UNIVERSITY OF CAMBRIDGE

physicalAI.build
PHYSICAL AI FOR THE BUILT WORLD

Physics-informed machine learning for civil engineers

British Council short course

Nandeesh Babanagar
University of Cambridge

Course plan

#	Block	Start	Min	Format	Notebook
1	Why physics-informed ML?	0:00	15	Lecture, short demo	N0, sections 1–2
2	How networks learn and differentiate	0:15	15	Lecture, short demo	N0, sections 3–4
3	Anatomy of a PINN	0:30	20	Lecture	N1, parts A and D
4	Forward PINN: the cantilever	0:50	35	Hands-on 1	N1
5	Break	1:25	10		
6	Inverse problems: back-analysis of the load	1:35	15	Lecture	N2 (preview)
7	Inverse PINN	1:50	30	Hands-on 2	N2
8	Failure modes of PINNs and remedies	2:20	30	Lecture, demos, concepts	N3
9	Wrap-up	2:50	10	Discussion	

By the end of today you will be able to

1. **Explain** why a data-only network struggles with sparse data, derivatives and extrapolation, and where PINNs sit among physics-based ML methods

2. **Explain** why PINNs use smooth activations and automatic differentiation, not finite differences

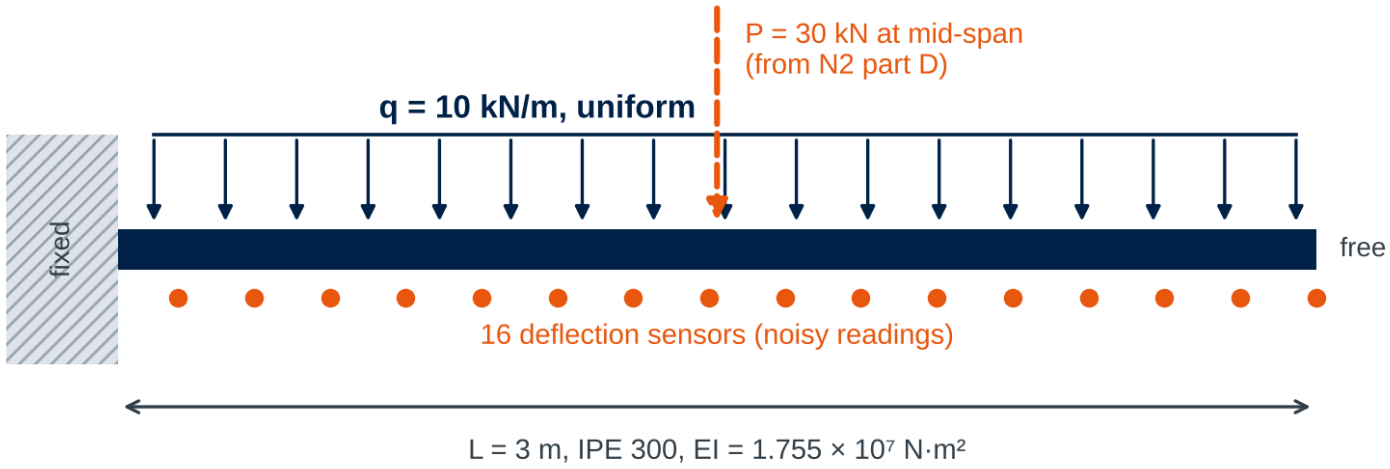
3. **Formulate** a PINN: residual, boundary and data losses, collocation points, non-dimensionalisation

4. **Build** a forward PINN for a cantilever in PyTorch, in a few lines, and check it against statics

5. **Build** an inverse PINN that recovers the load from sparse, noisy deflections

6. **Diagnose and fix** a PINN that fails on a point load, and recognise the other remedies and when neural operators are needed

Running case study: a cantilever under load



IPE 300, L = 3 m, EI = 1.755 × 10^7 N·m². Scaled units: 46 mm, 90 kN·m and 30 kN equal 1.

1 FAMILIAR

Cantilever action and an engineering deflection check.

2 VERIFIABLE

Exact solutions, so every result can be checked.

3 EXTENDABLE

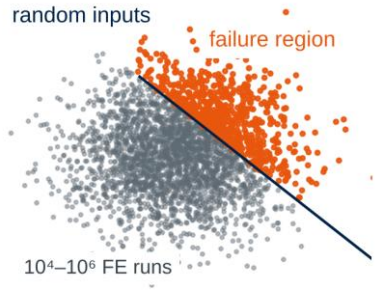
Same beam for sparse sensors, an unknown load, and a point load.

BLOCK 1

Why physics-informed ML?

Where a data-only network fails, and where physics can help. Demonstrations from Notebook N0.

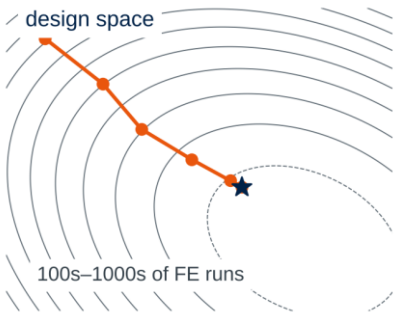
Engineering decisions: experiments, simulations, data



Reliability analysis

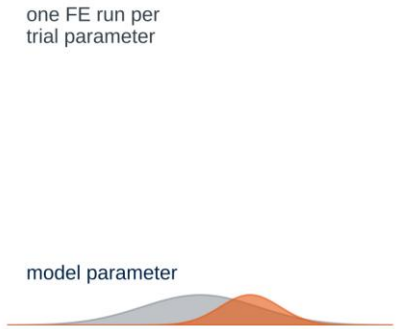
10^4 to 10^6 FE runs

100s of experiments



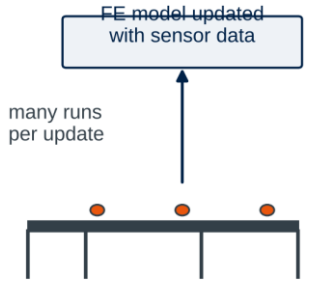
Design optimisation

100s to 1000s of candidates



Model updating

one FE run per trial



Digital twin

many runs per update



Real-time decisions

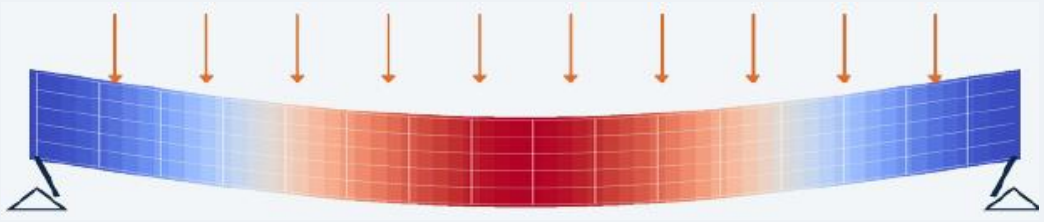
Experiments or FE too slow in seconds

Capabilities needed: rapid repeated evaluation · sparse-data use · physics consistency · uncertainty · interpretability

Physics-based simulation is powerful but slow and biased

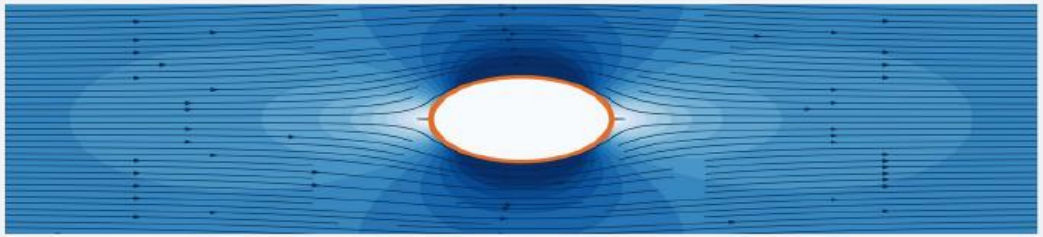
STRUCTURAL MECHANICS

Stress, deflection and stability



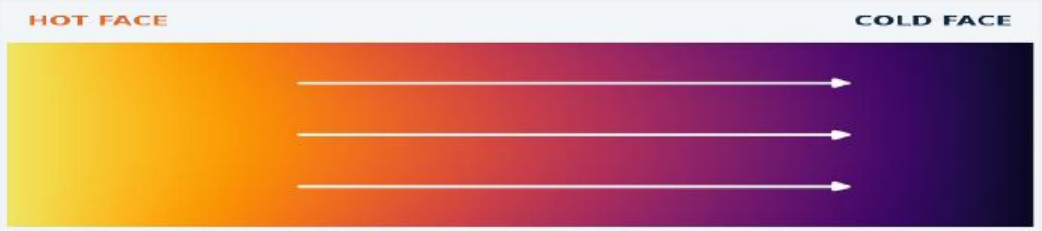
FLUID MECHANICS

Flow, pressure and transport



HEAT TRANSFER

Temperature fields and thermal loads



MATERIAL BEHAVIOUR

Heterogeneity, damage and failure



The challenge is often not one simulation, but thousands of runs, calibration and uncertainty.

Where many runs, or long tests, become the bottleneck

STRUCTURES



Structural digital twin

Update a beam model from sensor data without re-running the full FE model.

updates as data arrive

Sahin et al. 2024 [1]

GEOTECHNICAL



Embankment on soft clay

When can we pave? Early pore-pressure readings predict years of consolidation.

years of monitoring

Gong et al. 2024 [2]

WATER RESOURCES



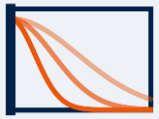
Subsurface transport

Infer conductivity and concentration from a few wells; forward models are costly to repeat.

few wells, many unknowns

He et al. 2020 [3]

MATERIALS



Chloride diffusion in concrete

Estimate the diffusion coefficient from limited data, not a long test.

long tests

Huang et al. 2026 [4]

CONSTRUCTION



Early-age hydration heat

Predict the temperature rise early, to control thermal cracking.

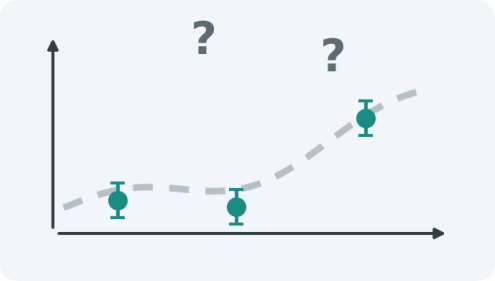
early-age prediction

Rahman et al. 2024 [5]

Same pattern in each: known physics, slow or costly to repeat, few measurements.

Four challenges motivate physics-enhanced ML

01

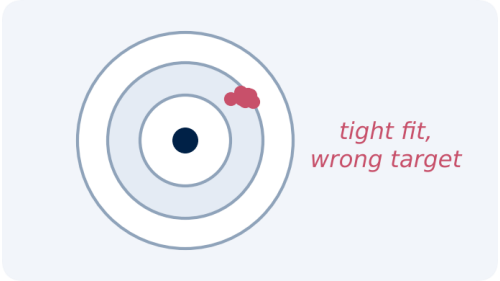


Limited useful data

Sensors can be sparse, noisy or uninformative.

IN OUR BEAM
16 noisy deflection readings, not the whole field (N0).

02

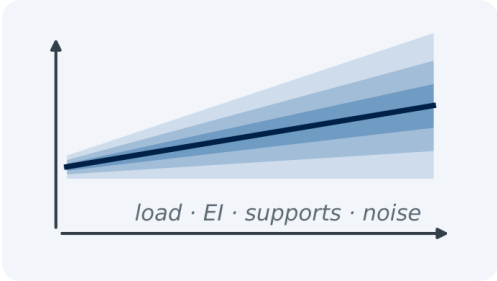


Accurate but wrong

A good numerical fit can still be physically implausible.

IN OUR BEAM
$R^2 = 0.975$ on deflection, yet the fixed-end moment is 62 % off (median of ten draws).

03

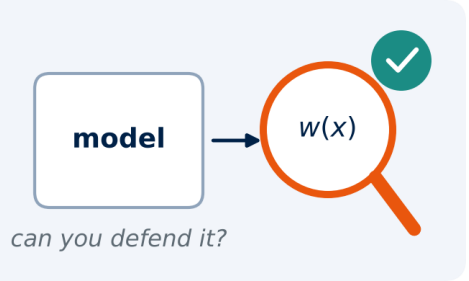


Uncertainty

Loads, parameters, model form and measurements are imperfect.

IN OUR BEAM
On a real wall the load itself is unknown (Block 6).

04



Explainability

High-consequence predictions must be defensible.

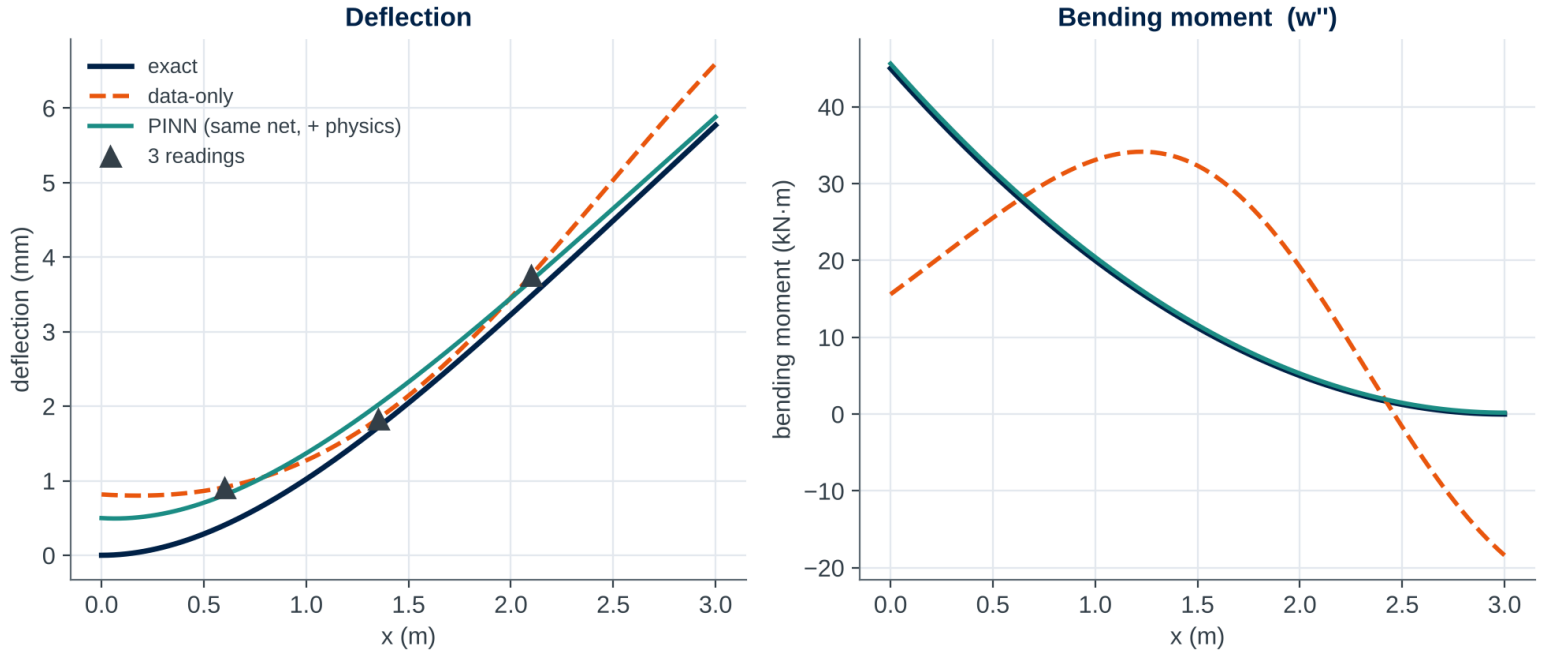
IN OUR BEAM
Check the PINN against statics: tip deflection $q L^4 / 8EI$.

Physics-enhanced ML adds knowledge where data alone are not enough.

Adapted from Cicirello (2024), Journal of Physics: Conference Series 2909, 012034.

Where data-only surrogates fall short

Our beam problem



Uniform load on a cantilever (L = 3 m, IPE 300, q = 10 kN/m). Network 1 → 30 → 30 → 1 (tanh), trained on three noisy deflection readings (▲, 5 % noise); the PINN adds the beam equation and the support conditions.

Median data-only run of 5 initialisations, and the PINN from the same start (N1 part E). Over the 5 runs: deflection error 15–17 % (data-only) and 5–11 % (PINN); moment error 65–81 % and 1.9–3.6 %.

DATA-ONLY NETWORK

17 % deflection error

73 % bending-moment error

- Tied to the sensor locations it has seen
- Nothing says the support holds the beam rigid

SAME NETWORK + PHYSICS

11 % deflection error

1.9 % bending-moment error

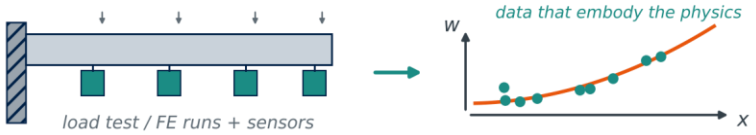
- The beam equation gives free labels everywhere
- Supports are enforced, not hoped for

Physics fills in what the sensors cannot see.

Where can bias enter a physics-enhanced model?

Observational bias

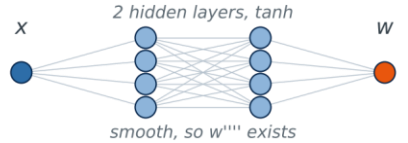
Introduce data that embody the physics



- Deflection readings from a lab load test or FE runs already obey beam physics
- N0: 16 noisy readings, no physics: deflection fits, moment about 62 % off

Inductive bias

Build prior assumptions into the network



optional hard constraint

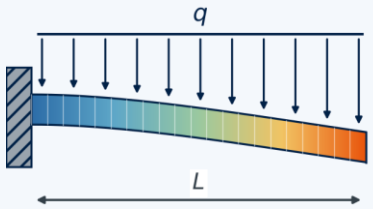
$w(x) = x^2 N(x)$

$w(0) = w'(0) = 0$ exactly

- Smooth tanh activation, because the equation needs w'''' (ReLU gives zero)
- One input x, one output w(x); no mesh
- Option: $w = x^2 \cdot N(x)$ fixes the clamped end exactly (not used in the notebooks)

Physics-enhanced model

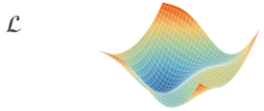
(e.g. PINN)



Predict w(x), q, EI …

Learning bias

Steer the training with physics



we choose the weights

$$\mathcal{L} = w_P \mathcal{L}_{phys} + w_D \mathcal{L}_{data} + w_B \mathcal{L}_{BC}$$

- Loss = beam-equation residual + boundary terms + data misfit
- We choose the weights (0.4 / 0.4 / 0.2), 100 collocation points, Adam
- Extra points near the point load help (Block 8)

Model-form / discrepancy bias

Use what we know of the governing equation

known physics: $EI \frac{d^4w}{dx^4} = q$ + discrepancy (optional): $\delta(x)$?

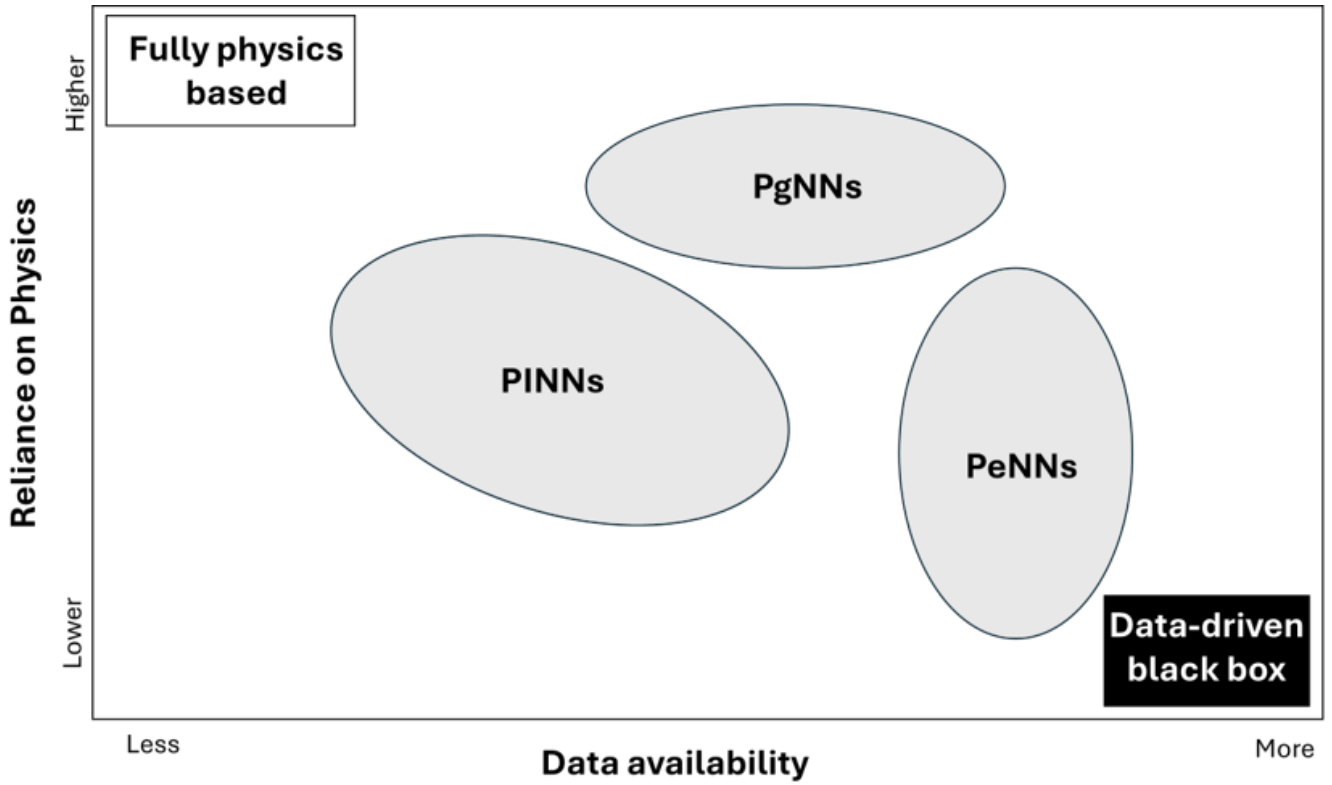
- ☑ Linear elastic
- ☑ Slender beam
- ☑ Small deflections

- Known form: EI w'''' = q (linear elastic, slender, small deflections)
- Partly known: q or EI recovered from data (Notebook 2)
- A discrepancy term δ(x) could absorb shear or soft supports (not tested here)

Key message: physics enters in four places, not only the loss; each bias narrows the space of admissible solutions.

Adapted from Cicirello (2024), Physics-Enhanced Machine Learning: a position paper for dynamical systems investigations, J. Phys.: Conf. Ser. 2909, 012034 (RASD 2024); arXiv:2405.05987.

How much physics, how much data?



PHYSICS-GUIDED (PGNN)

physics enters through the data: train on simulator output

G(x; μ) → infer μ

PHYSICS-ENCODED (PENN)

physics built into the architecture, e.g. a conserving layer

Gknown(x) + δθ(x)

PHYSICS-INFORMED (PINN)

the governing equation sits in the loss: works with little or no data

wθ(x), minimise Ldata + Lphysics

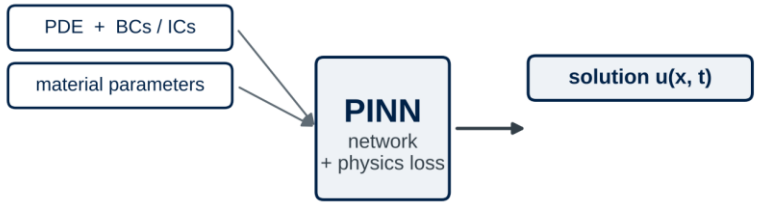
Today: PINNs, where the physics enters through the loss function.

After Faroughi et al., J. Comput. Inf. Sci. Eng. 24 (2024) 040802

Two problems PINNs address

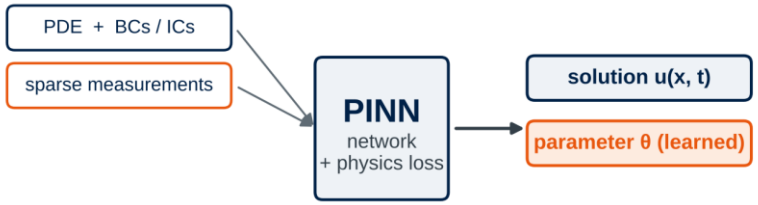
N1 (forward) · N2 (inverse)

Forward: predict the field



- PDE, boundary conditions and parameters are known.
- Cantilever: predict w(x) under a known load (Notebook 1).
- Consolidation, elasticity, composite curing (Bekele 2021; Haghighat et al. 2021; Niaki et al. 2021).

Inverse: infer the unknown



- PDE and sparse measurements are known.
- Cantilever: recover the load q from 16 noisy deflections (N2); q and a point load P together is harder (N3).

Raissi, M., Perdikaris, P. and Karniadakis, G.E. (2019). J. Comput. Phys. 378, 686–707.

BLOCK 2

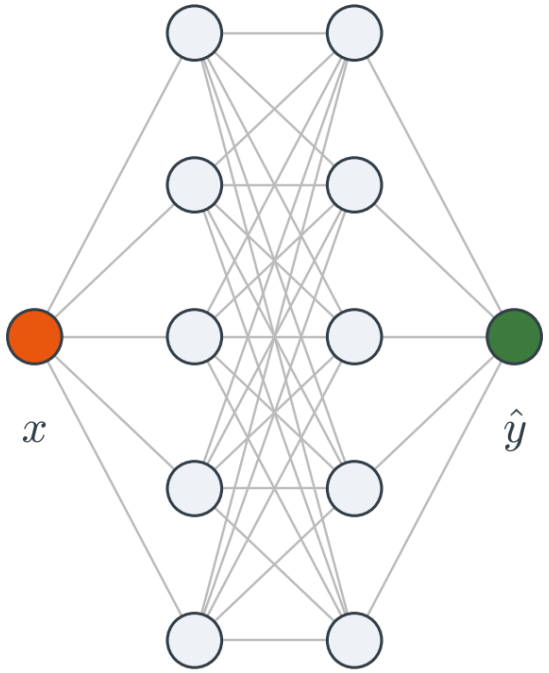
How networks learn and differentiate

A network is a smooth function with parameters. A PINN needs its derivatives with respect to the input, up to the fourth.

A network is a function with parameters

N0 · N1

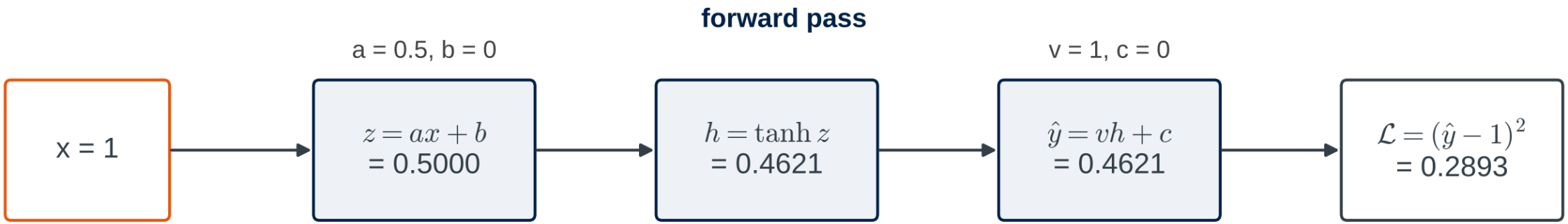
hidden layers: $h = \sigma(Wx + b)$



θ = all weights W and biases b (here 46 numbers)

- Input x, output w_θ(x): a function with parameters θ.
- Learning means adjusting θ to reduce a loss.
- Cantilever PINN: one input, two hidden layers of 30 tanh units, one output.
- That is 1,021 trainable parameters.
- Smooth activations matter: we differentiate the output four times.

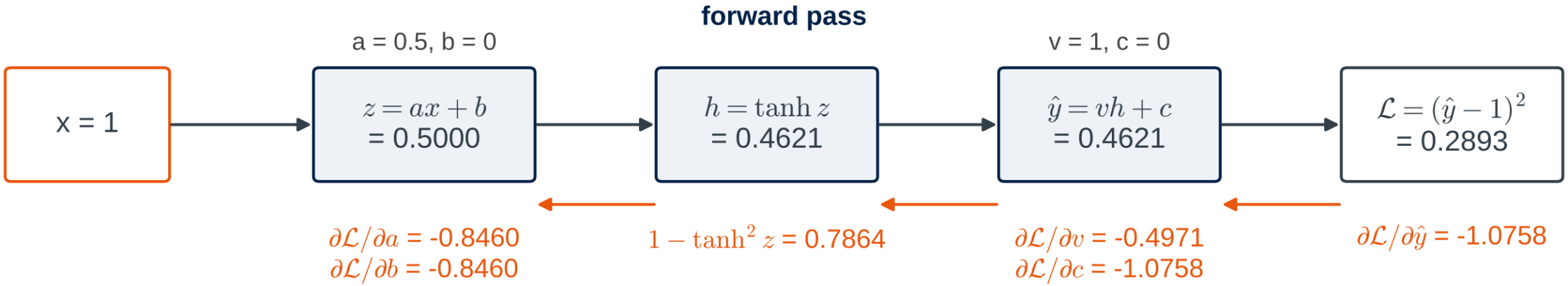
Forward pass: one neuron



Each layer applies a linear map and then an activation. Evaluating the network is the forward pass.

Backpropagation and gradient descent

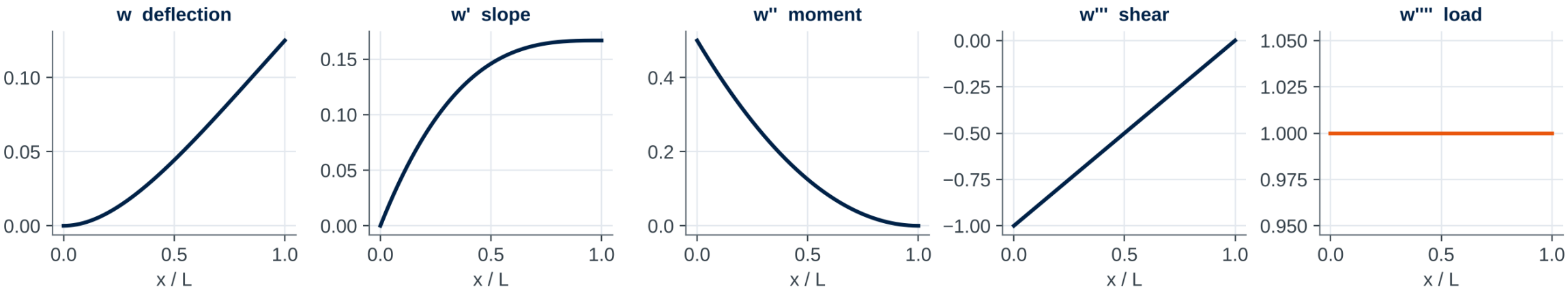
N0 · N1 training loop



- The loss is differentiated with respect to every weight using the chain rule.
- One backward pass gives all the gradients.

- Gradient descent updates $\theta \leftarrow \theta - \eta \nabla L$.
- Adam adapts the step size for each parameter.

Derivatives with respect to the input




```
def d(y, x):           # dy/dx by autograd
    return torch.autograd.grad(y, x, torch.ones_like(y),
create_graph=True)[0]

w1 = d(w, x); w2 = d(w1, x)
w3 = d(w2, x); w4 = d(w3, x)
```

- Differentiate the output with respect to the input x, not the weights.
- Each extra derivative is one more line (create_graph=True).
- The beam equation needs w'''' at every collocation point (N0 TODO 1).

Finite differences: simple, but the step size matters

Forward difference

$f'(x) \approx [f(x+h) - f(x)] / h$

error proportional to h (first order)

Central difference

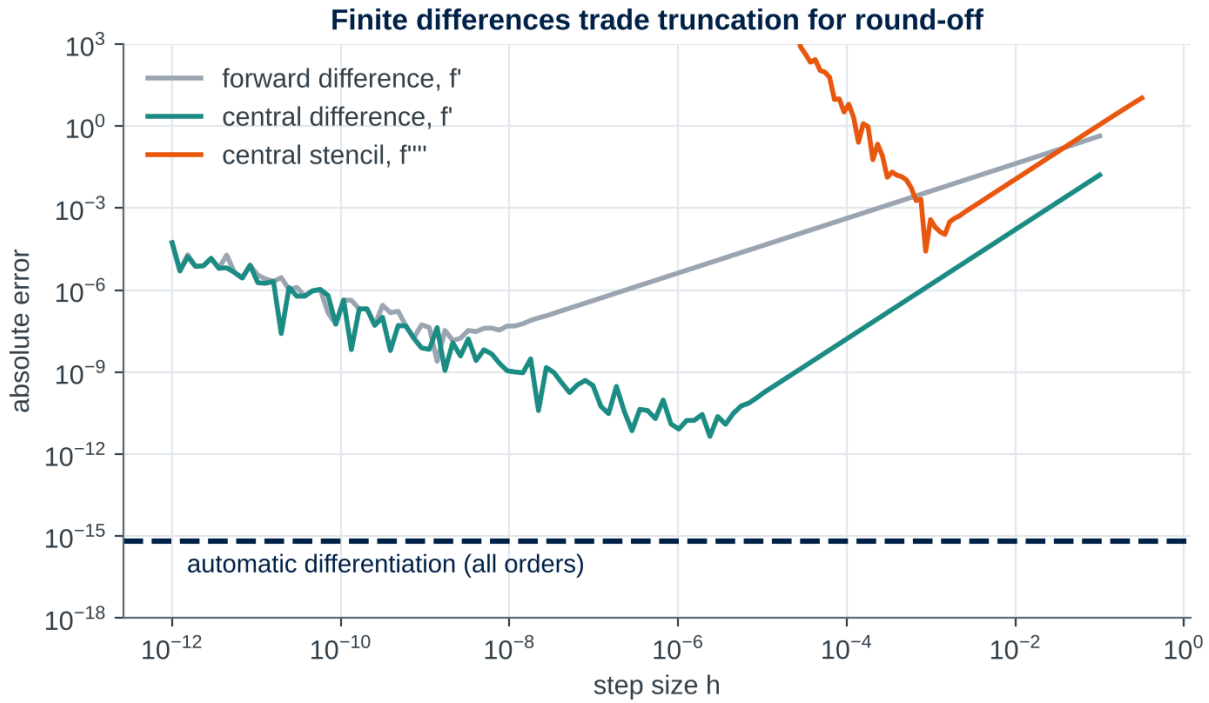
$f'(x) \approx [f(x+h) - f(x-h)] / (2h)$

error proportional to h^2 (second order)

Fourth derivative (central stencil)

$f''''(x) \approx [f(x+2h) - 4f(x+h) + 6f(x) - 4f(x-h) + f(x-2h)] / h^4$

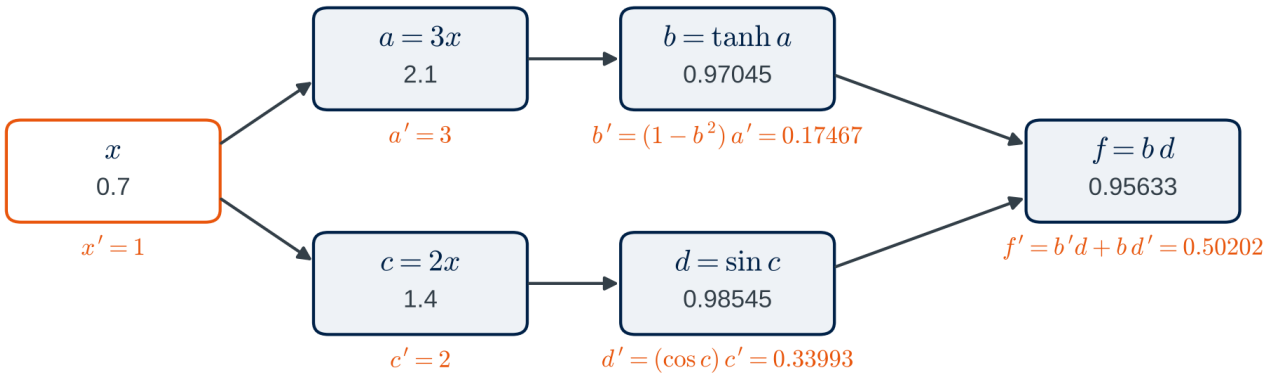
round-off error grows like $1 / h^4$



The fourth derivative is worse: best error 1×10^{-4} at $h = 1 \times 10^{-3}$. Autodiff: about 1e-15 at every order.

Test: f = sin 3x at x = 0.4 (N0 section 3). Best central difference: 2×10^{-11} at $h = 1 \times 10^{-6}$. N0 TODO 2.

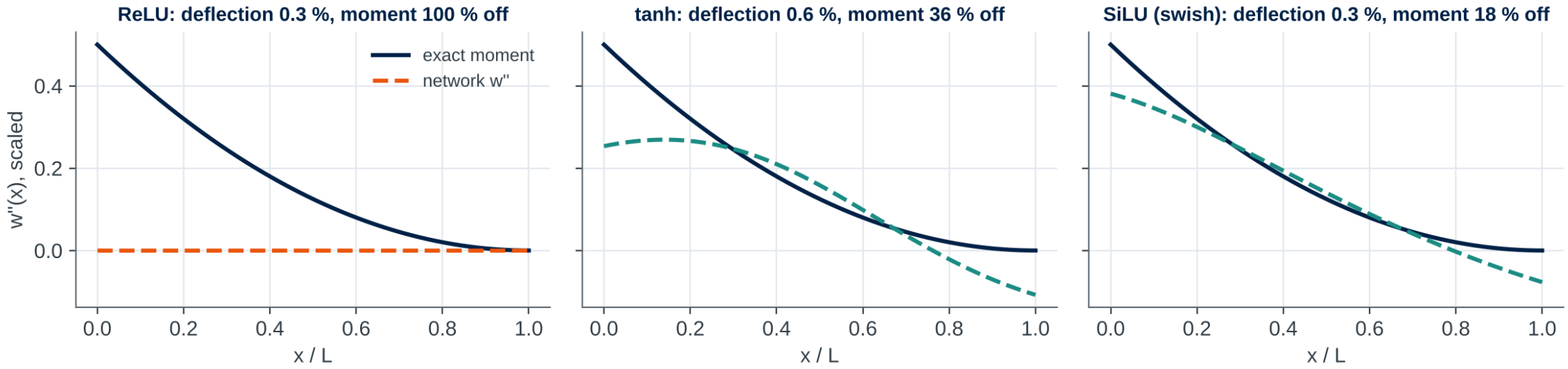
Automatic differentiation



- The network is built from simple operations. Autograd records them and applies the chain rule to each.
- Exact up to machine precision: 1e-15 or better at every order in this test.
- No step size, no truncation error. Differentiate again for higher orders.

Activations: smooth enough for four derivatives

N0 · exercise



Activation	Smooth enough for w""?
ReLU	No: w'' is zero
tanh	Yes
Swish / SiLU	Yes
sin (SIREN)	Yes

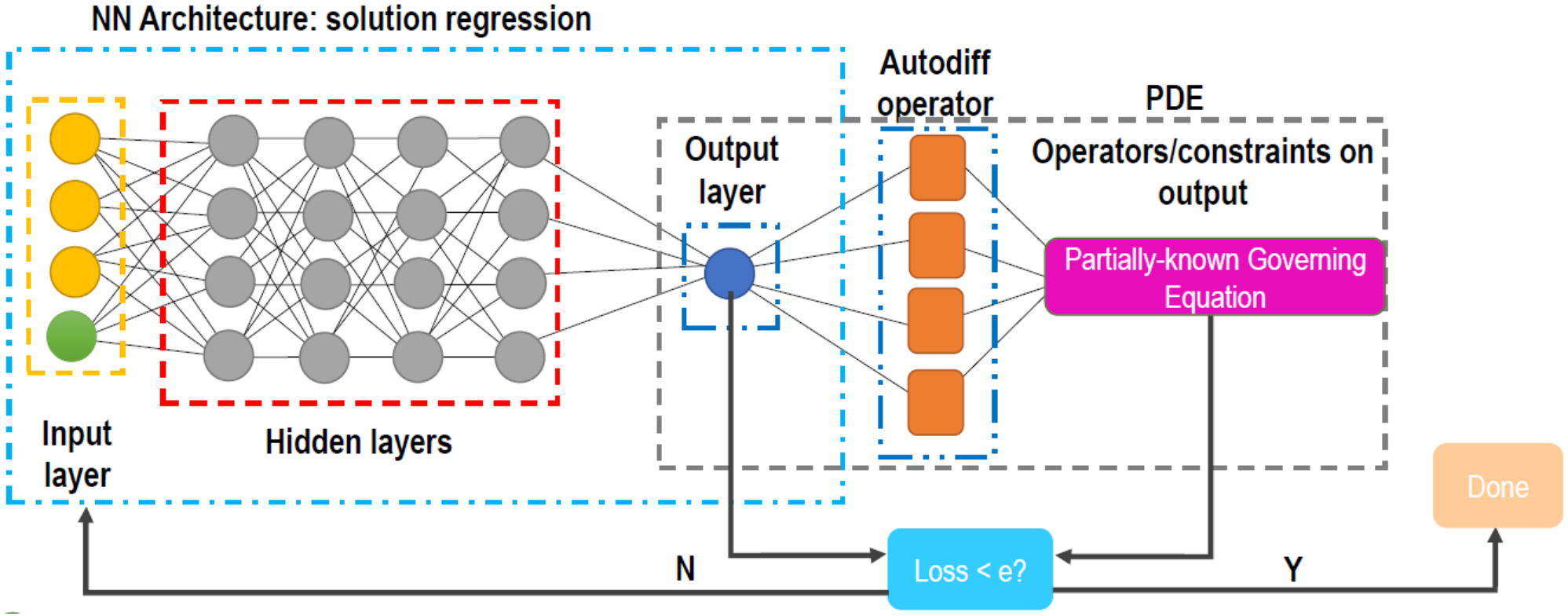
We use tanh: smooth and bounded. ReLU has zero second and fourth derivatives.

BLOCK 3

Physics-Informed Neural Networks

Loss design · collocation · non-dimensionalisation · results · strengths, limits and recent remedies.

PINNs idea



- **Observational biases**: introducing data that embody underlying physics and/or using data augmentation
- **Inductive biases**: incorporate prior assumptions and physical constraints in the architecture
- **Learning biases**: choice of loss functions, optimisation algorithm
- **Discrepancy biases**: incorporate terms of the partially-known governing equation

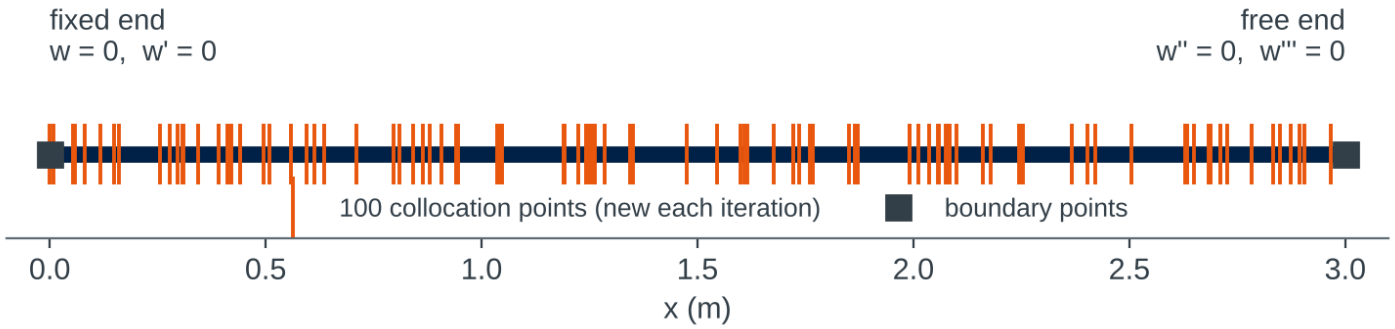
Steps for setting up PINNs workflow

N1 · training cell

1. Formulate the PDE + unknowns
2. Nondimensionalise
3. Choose architecture + outputs
4. Sample points + observations
5. Construct residual + losses
6. Train and monitor
7. Validate physics and engineering quantities

Collocation points and boundary terms

N1 · training cell



- 100 random points inside the beam, new at every iteration.
- Fixed end: w = w' = 0. Free end: w'' = w''' = 0.
- Boundary conditions are soft: they are loss terms.
- No mesh, and no data.

$$w(0) = w'(0) = 0, \qquad w''(L) = w'''(L) = 0$$

The residual and boundary terms are evaluated at points, not on a mesh.

Non-dimensionalisation: scale before you train

N1 · part A and D

$$\hat{w}^{(4)} = \hat{q} + \hat{P}\,\delta\left(\hat{x} - \tfrac{1}{2}\right), \quad \hat{x} = \frac{x}{L}, \quad \hat{w} = \frac{EI\,w}{qL^4}, \quad \hat{P} = \frac{P}{qL}$$

Scale (IPE 300, L = 3 m)	Value	Unit
Deflection q L⁴ / EI	46 mm	w = 1
Moment q L²	90 kN·m	M = 1
Load q L	30 kN	P = 1

- With q = 10 kN/m and P = 30 kN, the scaled problem is simply ŵ'''' = 1 + 1·δ(x̂ − ½).
- In SI units the physics term is about 6 × 10¹³ at the start of training, against 1.3 when scaled (N1 part D).

Scaling decides whether one loss weight can balance the physics and boundary terms.

Soft constraints and loss weights

N1 part D · N2

Forward

$$\mathcal{L} = \mathcal{L}_{phys} + \mathcal{L}_{bc}$$

Inverse

$$\mathcal{L} = 0.4\,\mathcal{L}_{data} + 0.4\,\mathcal{L}_{phys} + 0.2\,\mathcal{L}_{bc}$$

FORWARD PROBLEM (N1)

A wide middle range works

λ = 0.1 to 1 gives 0.1–1.0 % error; λ = 0.001 gives 13–14 % and λ = 100 gives 3–7 % (3,000 iterations, two seeds).

INVERSE PROBLEM (N2)

Terms compete

Data, physics and boundary terms pull in different directions, so the weights matter. Balance them after scaling.

Weights 0.4 / 0.4 / 0.2 from the sensitivity analysis in Babanagar and Sheil (2026).

BLOCK 4

Hands-on 1: the forward PINN

35 min · Notebook N1 · student version; the solution version is for checking

Hands-on 1: Notebook N1

N1 · notebook

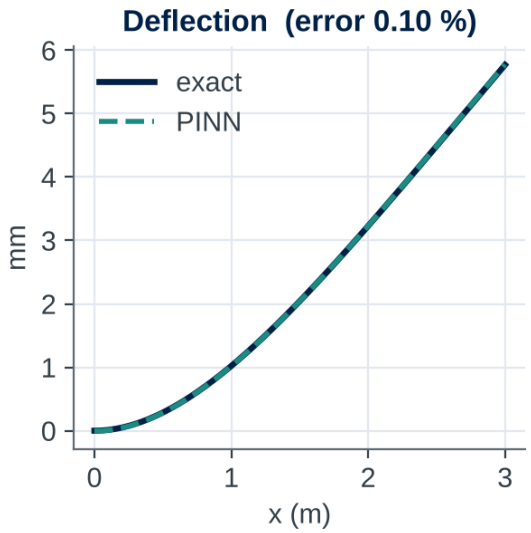
Task	What you do	Automatic check
TODO 1: derivatives	w', w'', w''', w'''' with the helper d(y, x)	$x^4/24$ has w'''' = 1
TODO 2: physics loss	mean of $(w'''' - 1)^2$	exact solution gives zero loss
TODO 3: boundary loss	four squared conditions, summed	exact solution gives zero loss
TODO 4: engineering values	tip deflection in mm, fixed-end moment in kN·m	within 5 % of 5.77 mm and 45.0 kN·m

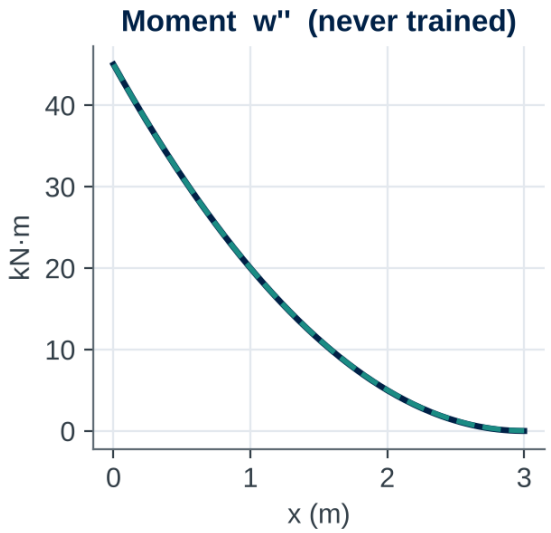
The heart of the notebook, in code

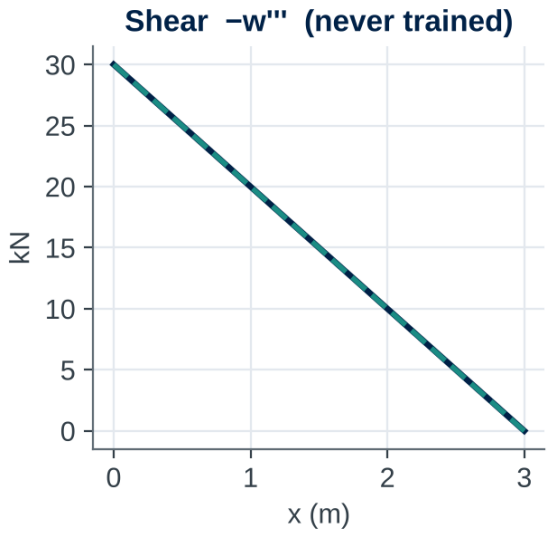
```
w1 = d(w, x);  w2 = d(w1, x);  w3 = d(w2, x);  w4 = d(w3, x)              # TODO 1
L_phys = ((w4 - 1)**2).mean()                                             # TODO 2
L_bc   = w[0]**2 + w1[0]**2 + w2[1]**2 + w3[1]**2                         # TODO 3: fixed end, free end
loss   = L_phys + L_bc;  opt.zero_grad();  loss.backward();  opt.step()
```

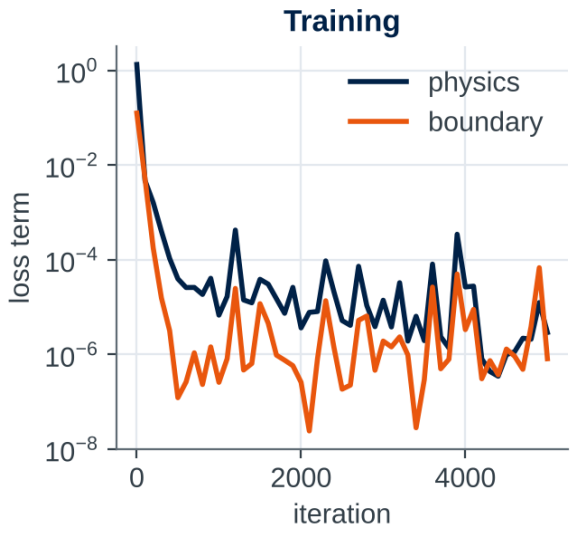
Forward PINN: training and result

N1 · part B









SET-UP

Network 1 → 30 → 30 → 1, tanh

Adam, learning rate 5e-3, 100 fresh collocation points per iteration, 5,000 iterations. No data.

RESULT

Error of the deflection: 0.10 % (seed 0)

Three seeds: 0.1–1.9 %. The moment and the shear, never trained on, also match the exact curves.

N1 part B, solution run.

Checking the result like an engineer

N1 · TODO 4

5.78 mm

Tip deflection, PINN

exact $q L^4 / 8EI = 5.77$ mm

45.0 kN·m

Fixed-end moment, PINN

exact $q L^2 / 2 = 45.0$ kN·m

0.10 %

Relative error of w

over the whole beam

- IPE 300 ($EI = 1.755 \times 10^7$ N·m²), L = 3 m, q = 10 kN/m. Scales: 46 mm for w = 1, 90 kN·m for M = 1.
- The moment is the second derivative of the network, and the PINN was never shown a moment.
- A data-only network gave a moment error of 62 % at the fixed end (median of ten draws, N0).

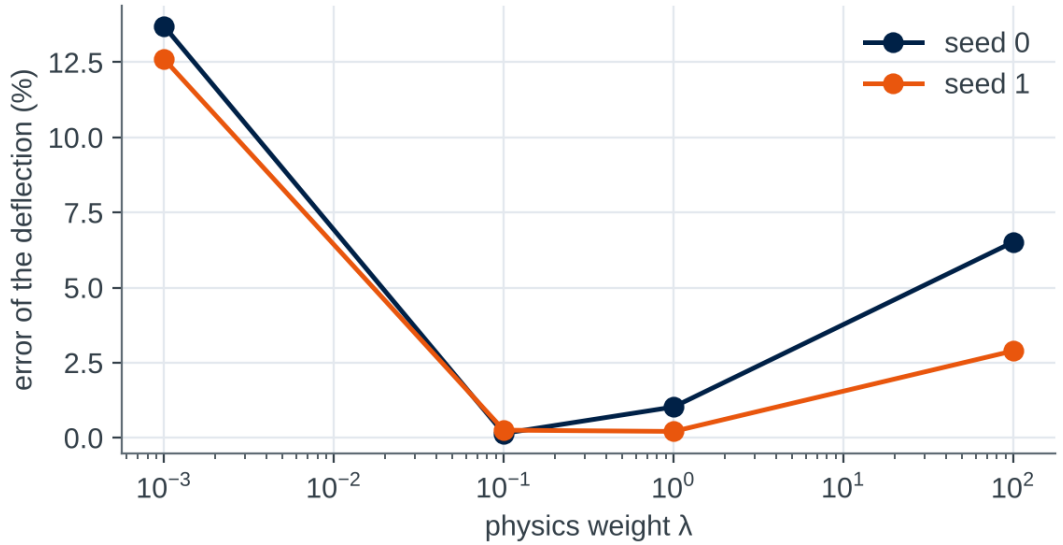
The PINN agrees with statics, and gives moment and shear for free.

Why scale, and how sensitive is the weight?

N1 · C1 and C2

Physics term at start of training	Value
SI units	6×10^{13}
Scaled	1.33

- In SI units the optimiser sees only the physics term.
- After 2,000 iterations in SI units the tip deflection is -35 mm instead of +5.8 mm.
- Scaling puts every term near one.



- Forward PINN: a wide middle range of λ works.
- λ = 0.1 and 1: 0.1–1.0 % error. Extremes: 13–14 % and 3–7 %.

Scale first. Weights matter most when terms compete, as in the inverse problem.

Break · 10 minutes

BLOCK 6

Inverse problems: back-analysis of the load

15 min · lecture and polls · the PINN as a back-analysis tool

What is unknown on a real wall?

Quantity	Known?	How we would get it
Geometry L, section, EI	Mostly	Drawings, material tests
Boundary conditions	Roughly	Design assumptions
Earth pressure q	No	Back-analysis of readings
Deflections w(x_i)	Yes, at a few points	Inclinometers, sparse and noisy

Unknown load, known physics, few readings: an inverse problem.

Forward and inverse problem

FORWARD

load q → deflection w(x)

Physics loss + boundary loss. No data. The load is a number in the equation.

INVERSE

readings w(x_i) → load q

Data loss + physics loss + boundary loss. The load is a trainable parameter.

$$\hat{q} = \exp(\log \hat{q}), \qquad \hat{P} = \exp(\log \hat{P}) \qquad \text{(trainable, always positive)}$$

- q is stored as log q, so that it stays positive: q = exp(log q).
- The same autograd machinery; only the loss changes.

The inverse loss

$$\mathcal{L} = 0.4\,\mathcal{L}_{data} + 0.4\,\mathcal{L}_{phys} + 0.2\,\mathcal{L}_{bc}$$

DATA

fit the readings

$(w(x_i) - \tilde{w}_i)^2$ / peak². Only at the sensors.

PHYSICS

obey the beam equation

$(w'''' - q)^2$ at fresh random points, everywhere.

BOUNDARY

respect the supports

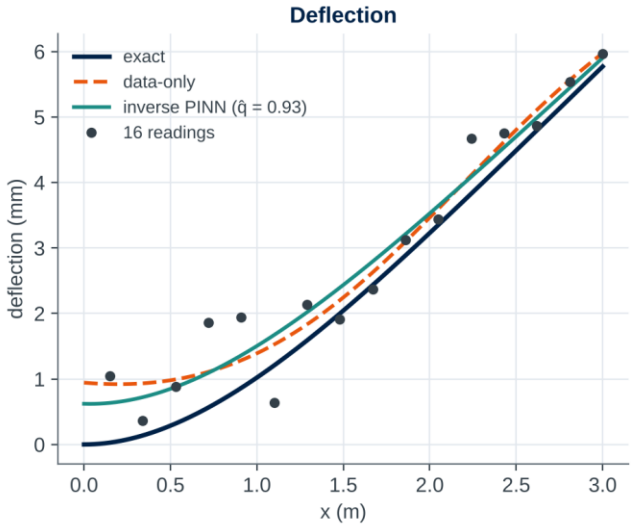
Fixed end and free end: four conditions.

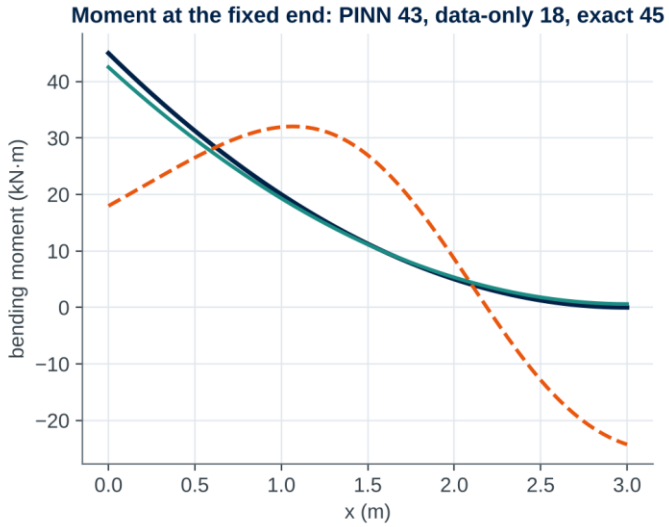
Weights 0.4 / 0.4 / 0.2 as in the paper; the load has its own learning rate.

Babanagar and Sheil, ISARC 2026.

Preview: PINN versus data-only

N2 · section B





q = 0.93

Recovered load (true 1.00)

7 per cent off with 10 % noise

43 vs 18 kN·m

Fixed-end moment, PINN vs data-only

exact 45 kN·m

- Same sensors, same noise, same network: the physics term is the only difference.

N2 part B, noise draw 0 (16 sensors, 10 % noise).

BLOCK 7 · 30 MIN

Hands-on 2: the inverse PINN

30 min · Notebook N2 · recover the load from sixteen noisy readings

Hands-on 2: Notebook N2

N2 · notebook

Task	What you do	Automatic check
TODO 1: the load	a trainable log q, starting at −1 (q = 0.37, not the truth)	requires_grad is true
TODO 2: data loss	mean of $((w(x_i) - \tilde{w}_i) / \text{peak})^2$	zero for exact readings
TODO 3: physics loss	mean of $(w'''' - \text{load})^2$	zero for the exact solution

```
log_q = torch.tensor(-1.0, requires_grad=True)       # TODO 1: q starts at 0.37
L_data = (((model(X) - Y) / Y.max())**2).mean()      # TODO 2
L_phys = ((w4 - load)**2).mean()                     # TODO 3, load = exp(log_q)
loss = 0.4*L_data + 0.4*L_phys + 0.2*bc_loss(model)
opt.zero_grad(); opt_load.zero_grad(); loss.backward(); opt.step(); opt_load.step()
```

Result: recovering the load

0.93

Recovered q (true 1.00)

16 sensors, 10 % noise

9.3 kN/m

In engineering units

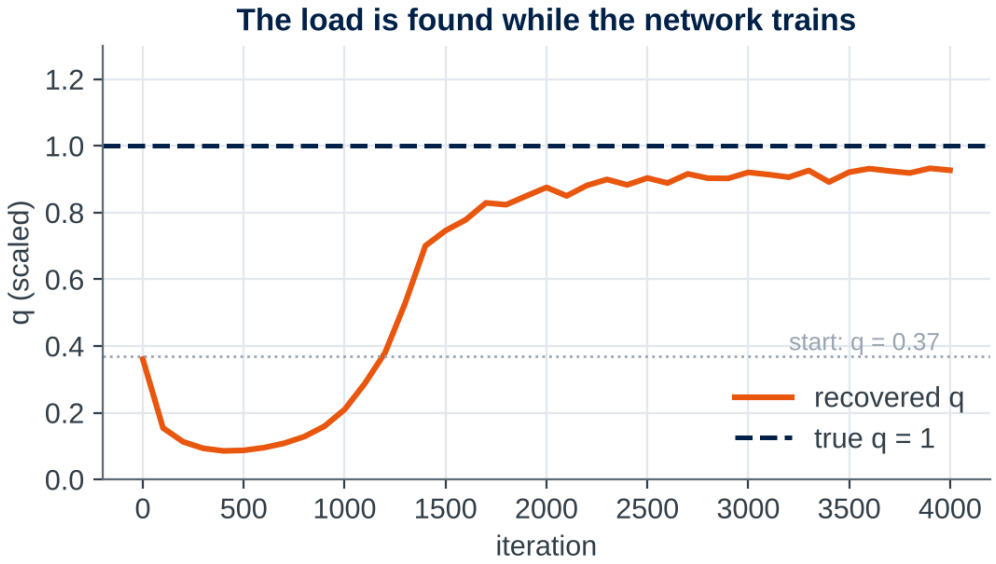
true 10 kN/m

4,000

Iterations

two optimisers, one loss

- The load enters the physics term; the data pull it towards the readings.
- The network and the load are found together, from the same loss.
- Check the result against the sensors, not only against the loss value.

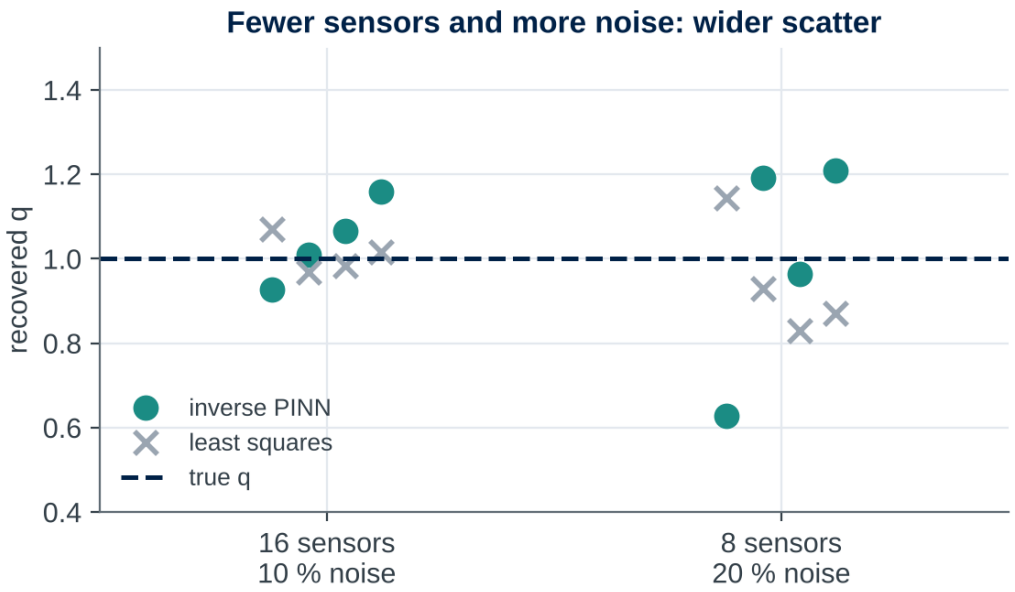


Class experiment: sensors or noise?

N2 · C1

Sensors, noise	PINN $\hat{q}$, four noise draws	Least squares
16, 10 %	0.93, 1.01, 1.07, 1.16	0.97–1.07
8, 20 %	0.63, 1.19, 0.96, 1.21	0.83–1.14
Your choice		

- Post your (sensors, noise, $\hat{q}$) on the chat.
- We changed both at once. Which runs would separate the effect of the sensors from the effect of the noise?
- Why is a single unknown so forgiving? What changes with two unknowns (N2 part D)?



N2 part C, solution run.

UNIVERSITY OF CAMBRIDGE

BLOCK 8

Failure modes of PINNs, and remedies

30 min · demonstrations from N3, then a short tour of remedies and neural operators (concepts only)

What goes wrong, and what people try

Concepts

Loss terms compete
data, physics and boundary terms differ in size

→ TRY
Adaptive weights (annealing, NTK, ReLoBRaLo) · self-adaptive PINNs

Points in the wrong places
the residual is large where points are few

→ TRY
Residual-based adaptive sampling (

Sharp features
point load, crack, interface: spectral bias

→ TRY
Domain decomposition (cPINN, XPINN; used in N3) · Fourier features · adaptive sampling

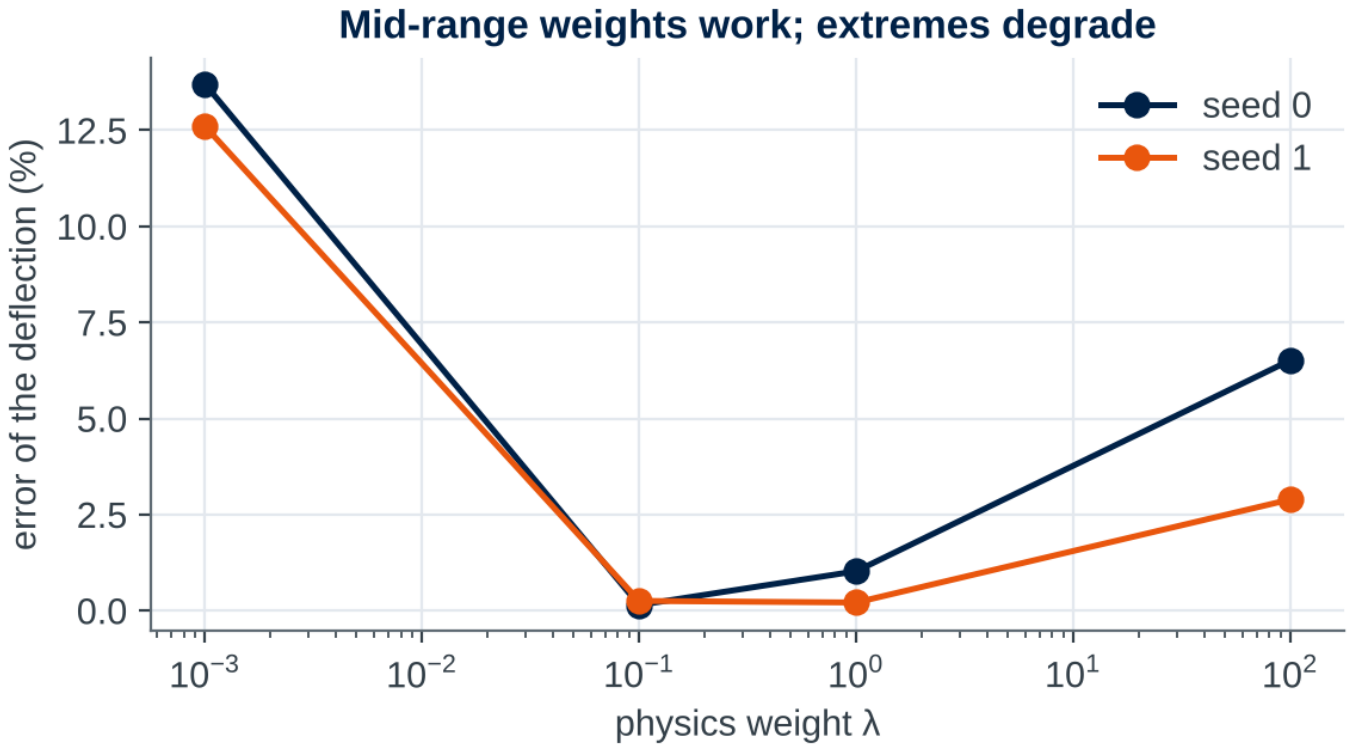
Boundary conditions fight the physics
a soft penalty competes with the equation

→ TRY
Hard constraints: build the boundary conditions into the network

A new load, a new training
a PINN solves one problem instance

→ TRY
Neural operators: DeepONet, FNO, PINO (end of this block)

Loss imbalance and gradient pathologies



What is happening

- λ small: the physics term is ignored (13–14 % error).
- λ large: the boundary terms are outvoted (3–7 %).
- A sweet spot exists (λ = 0.1 to 1: 0.1–1.0 %), but it moves with the problem.
- Gradients of different terms can differ by orders of magnitude (Wang et al. 2021).

Published remedies balance the terms automatically: see loss balancing later in this block.

Loss balancing: let training choose the weights

Concept

Fixed weights

gradient size of each loss term

Adaptive weights

gradient size of each loss term



$$L = \lambda_{data} L_{data} + \lambda_{phys} L_{phys} + \lambda_{bc} L_{bc}$$

fixed λ: chosen once by hand (ours: 0.4 / 0.4 / 0.2) · adaptive λ: updated during training

Gradient-based

Scale each term so that its gradient has a similar size.

Learning-rate annealing: Wang, Teng & Perdikaris 2021 · GradNorm

Convergence-based

Give more weight to the term that is learning most slowly (neural tangent kernel).

Wang, Yu & Perdikaris 2022

Relative progress

Give more weight to the term whose loss has fallen least.

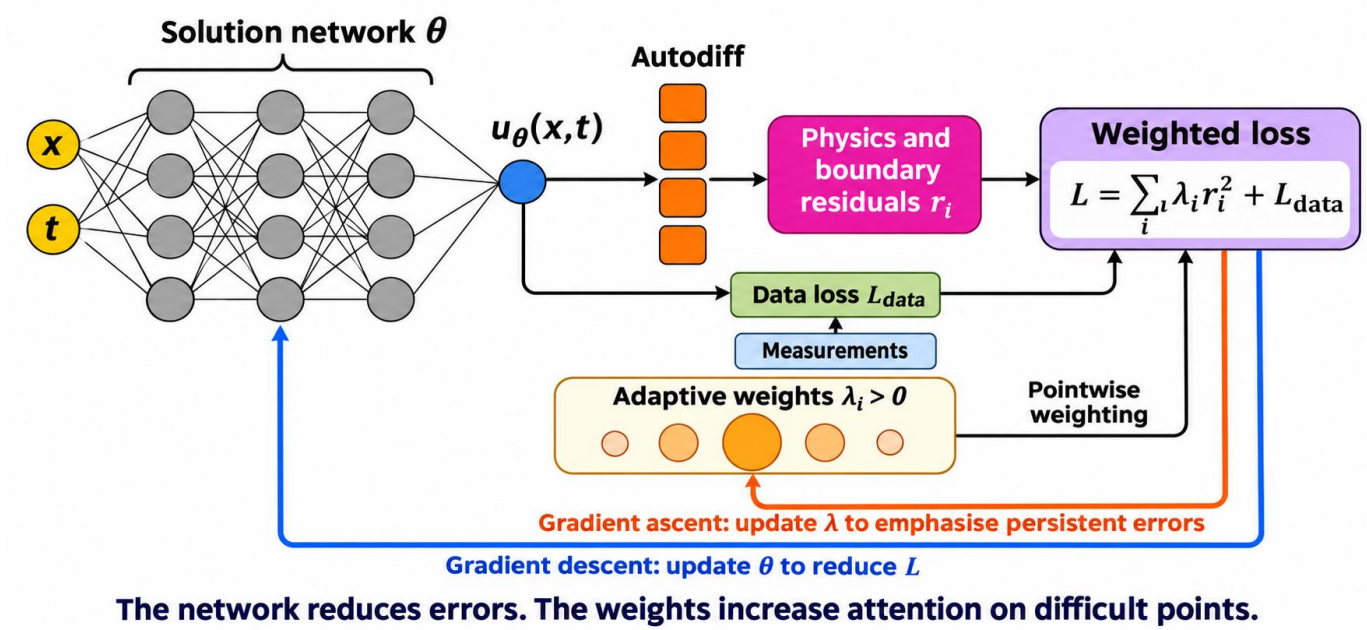
SoftAdapt · ReLoBRaLo: Bischof & Kraus 2025

Bars are a schematic of the idea, not a result. In N1 part D the middle range λ = 0.1 to 1 worked and both extremes failed.

The weights become part of the training, not a number to tune by hand.

Self-adaptive PINNs: a weight for every point

Concept



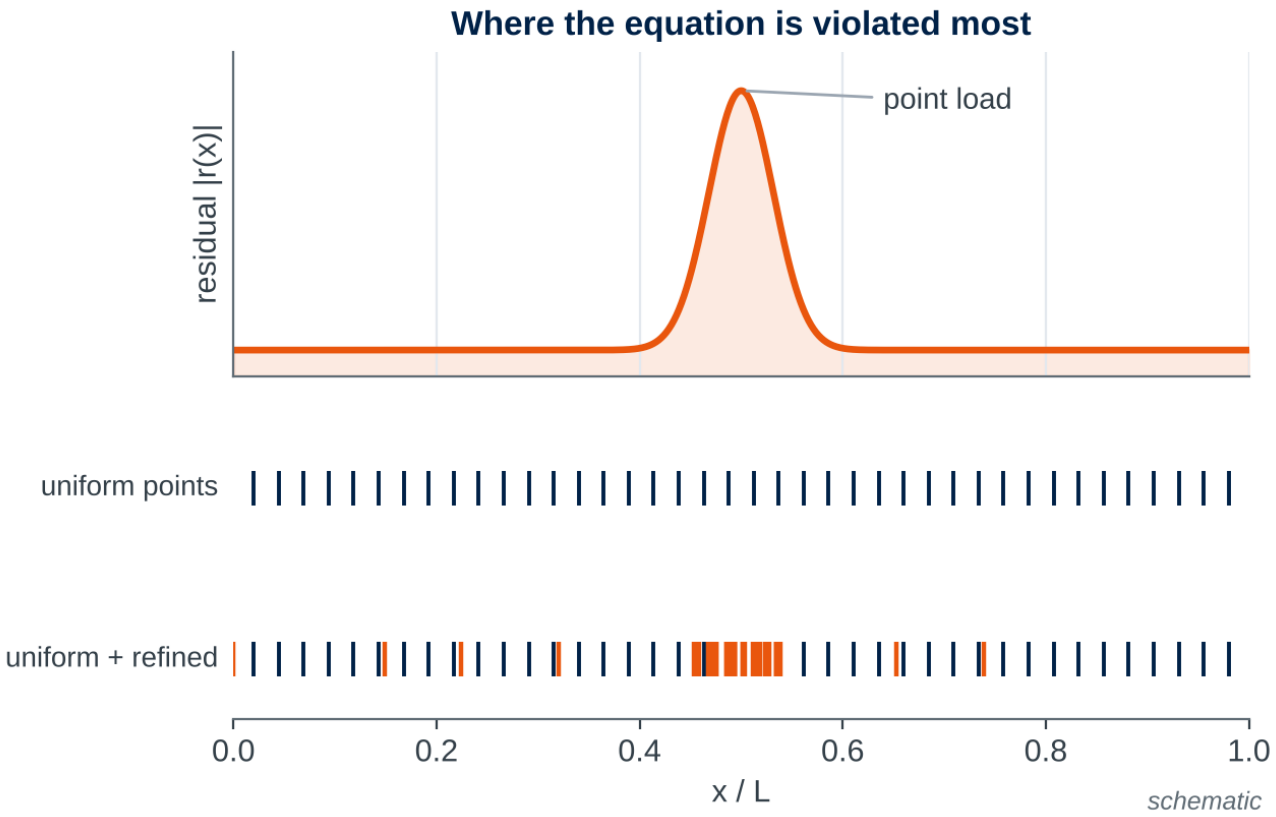
- **Predict the solution** while fitting data and satisfying physics.
- **Increase loss weights** at points with persistent errors using gradient ascent.
- **Update the network** using gradient descent, giving difficult points more attention.

McClenny & Braga-Neto (2023). Self-adaptive PINNs, J. Comput. Phys. 474, 111722.

The network tells you where it is struggling; you do not have to guess.

Adaptive sampling: points where they matter

Concept



1. Train on uniform collocation points.
2. Evaluate the residual on many candidate points.
3. Add points where it is large (RAR), or draw new points in proportion to it (RAD).
4. Retrain; repeat.

Schematic. Lu et al. 2021 (RAR, DeepXDE); Wu et al. 2023 (RAD, RAR-D).

Better points help, but they do not remove spectral bias on their own.

Features and constraints built into the network

Concept

Fourier features: a sharper input



- The network starts with high-frequency building blocks, so a spike is easier to build.
- The scale of b_k matters: too small and the output stays smooth, too large and the network fits noise.
- **Not tested here.** Tancik et al. 2020; for PINNs, Wang, Wang & Perdikaris 2021.

Hard constraints: conditions by construction

$$w(x) = x^2 \cdot N_\theta(x)$$

- The clamped end holds exactly for any weights: $w(0) = 0$ and $w'(0) = 0$. Two loss terms disappear.
- The free-end moment and shear stay in the loss, or come out naturally in the weak form (next slide).
- **Cost:** the function must be built for each geometry. Sukumar & Srivastava 2022 (their examples include beam bending).

Build in what you know exactly; let the loss handle the rest.

Weak form: lower the derivative order

Concept

Strong form (this course)

$$EI\, w''''(x) = q(x)$$

needs the 4th derivative of w

integrate by parts, twice →

Weak form (V-PINN)

$$\int_0^L EI\, w'' v''\, dx = \int_0^L q\, v\, dx$$

needs only the 2nd derivative of w and v

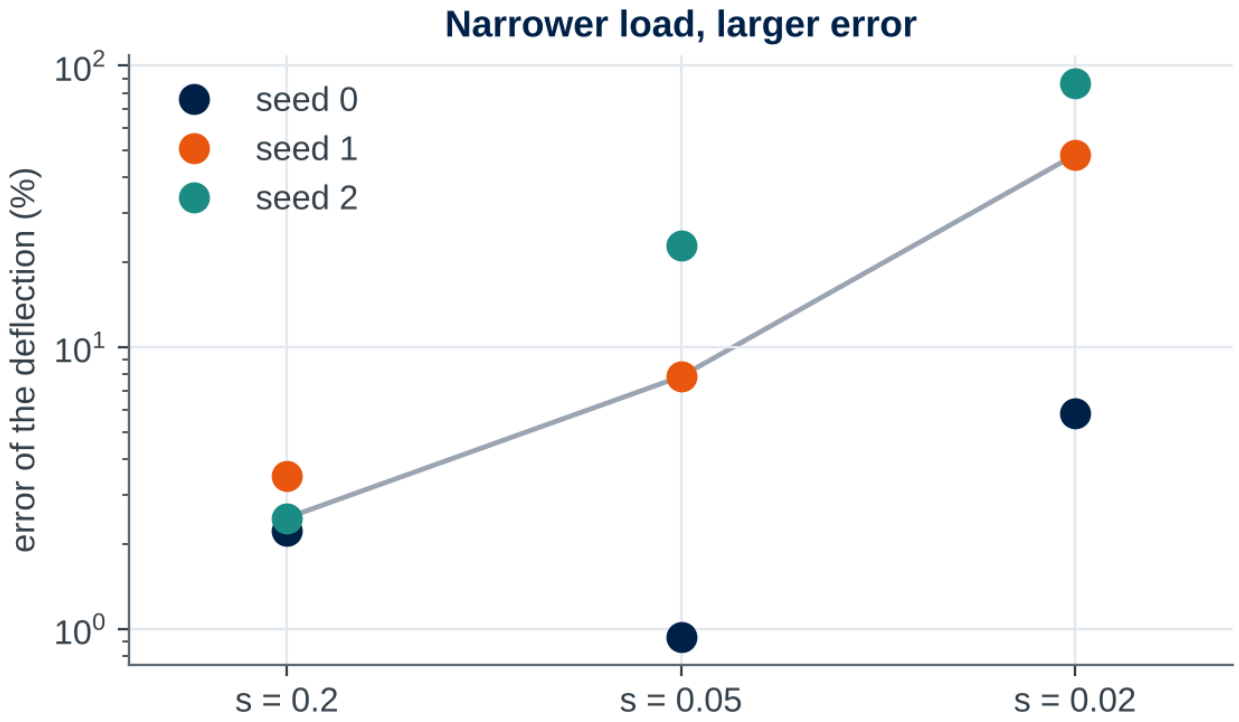
- Multiply the beam equation by a test function v(x) and integrate: the same move as the virtual-work method already used in finite element beam elements.
- Integrating by parts twice moves two derivatives off w and onto v, so training only ever needs the second derivative, not the fourth.
- The free end's moment and shear conditions can fall out of the integration by parts itself, as natural boundary conditions, instead of being penalised in the loss.
- The cost moves to choosing test functions and integrating them accurately, which is why this course's notebooks stay with the strong, collocation form.

Kharazmi, Zhang & Karniadakis (2021). hp-VPINNs, Comput. Methods Appl. Mech. Eng. 374, 113547.

A different loss, not a different network: same network, same autodiff machinery.

A point load breaks the simple PINN

N3 · Lab 1, N2 · C2



q = 1.7–1.9, P = 0.01

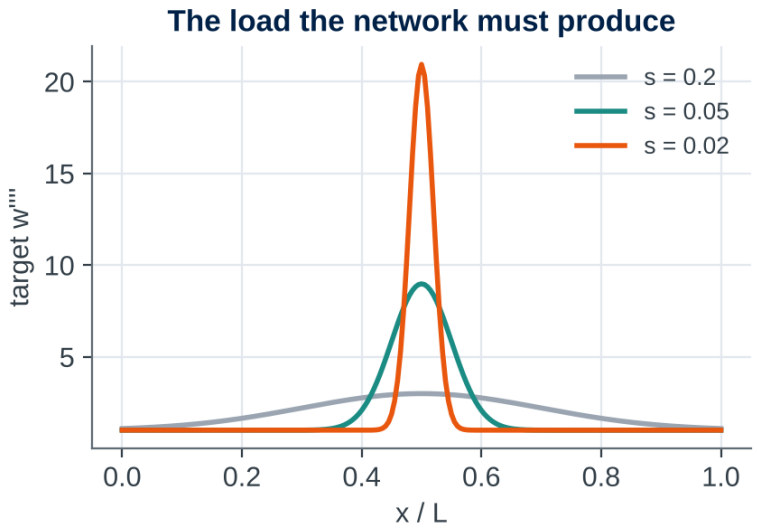
Single-network inverse PINN (N2 part D)

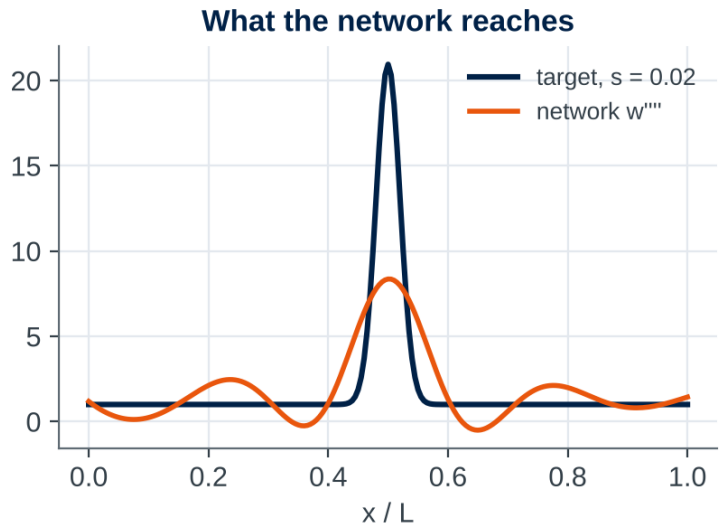
true values 1.00 and 1.00

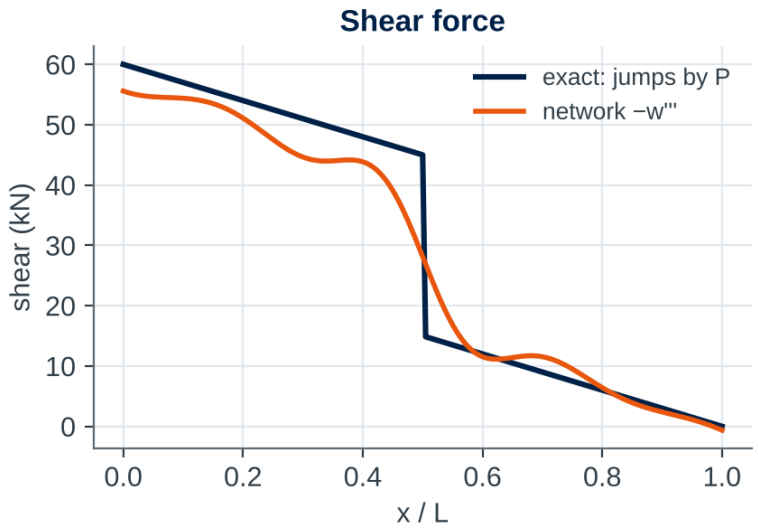
- Forward: the error grows as the load gets narrower (s = 0.02: 6–87 %).
- Inverse: the deflection is fitted well, but the split between q and P is wrong.
- A good fit to the data is not proof that the parameters are right.

N3 Lab 1 (three seeds per width) and N2 part D.

Why: the spike, and spectral bias





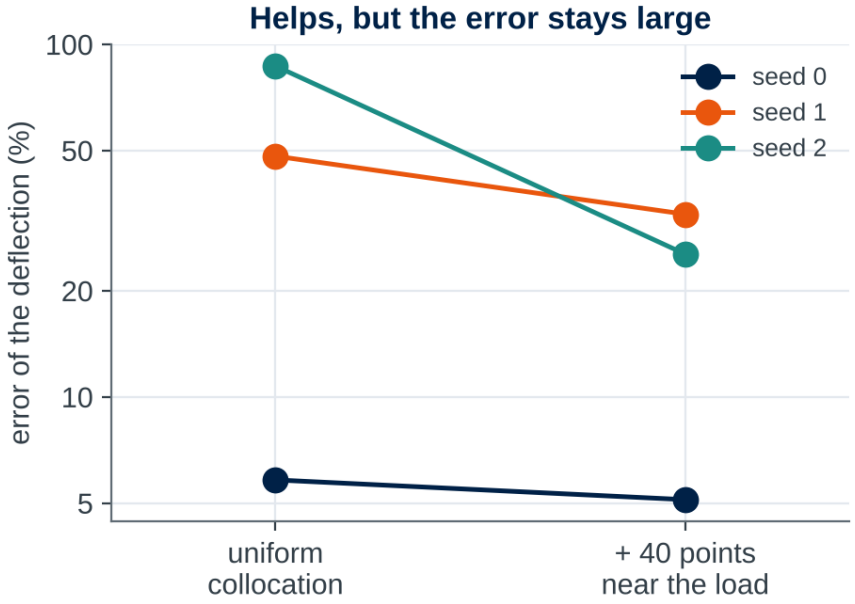


- w'''' must contain a narrow spike at x = ½.
- The shear force w''' jumps at the load.
- Networks learn smooth functions first: spectral bias.
- More iterations help only slowly.

Rahaman et al. 2019; Wang, Wang and Perdikaris 2021 (see references).

Simple fixes

N3 · Lab 2



6–87 %

Single network, s = 0.02

uniform collocation, 3 seeds

5–33 %

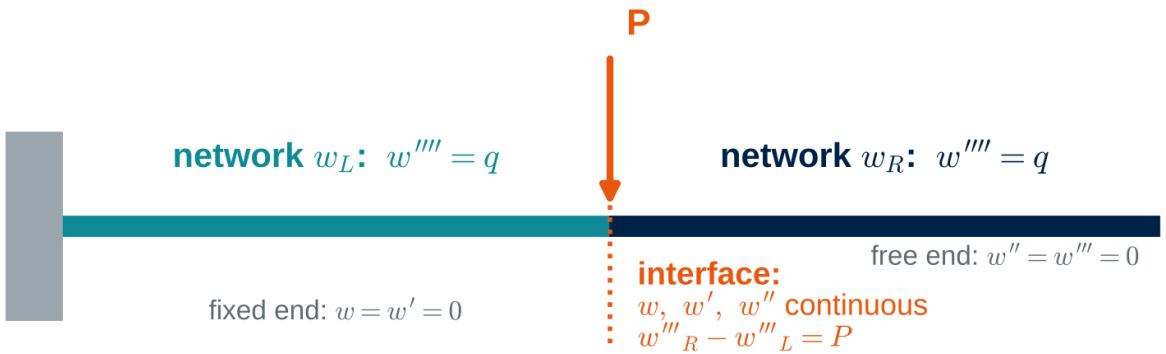
With 40 points near the load

lower, but still large

Remedy	Idea	Our experience
Refine collocation	More points where the load is	Lowers the error, does not remove it (N3 Lab 2)
Fourier features	Sharper input basis	Not tested here
Adaptive weights / RAR	Re-weight or add points automatically	Not tested here

N3 Lab 2 (three seeds).

Domain decomposition: load as interface



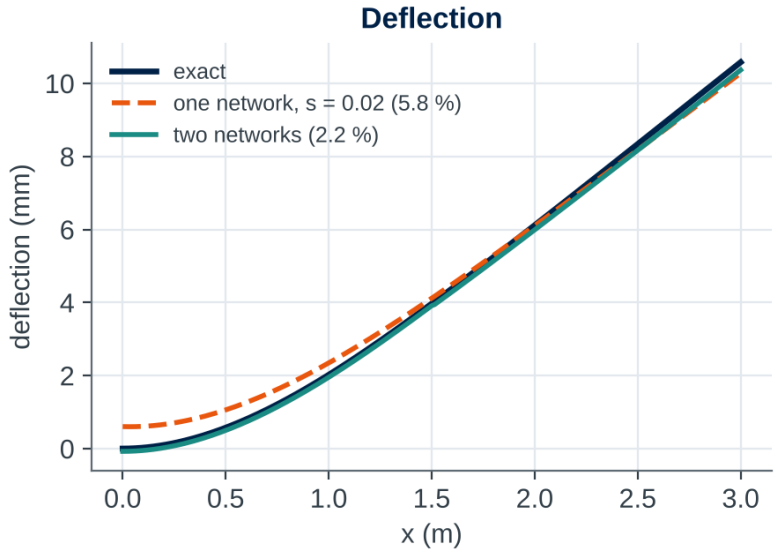
$$w_L = w_R, \quad w'_L = w'_R, \quad w''_L = w''_R, \quad w'''_R - w'''_L = \hat{P}$$

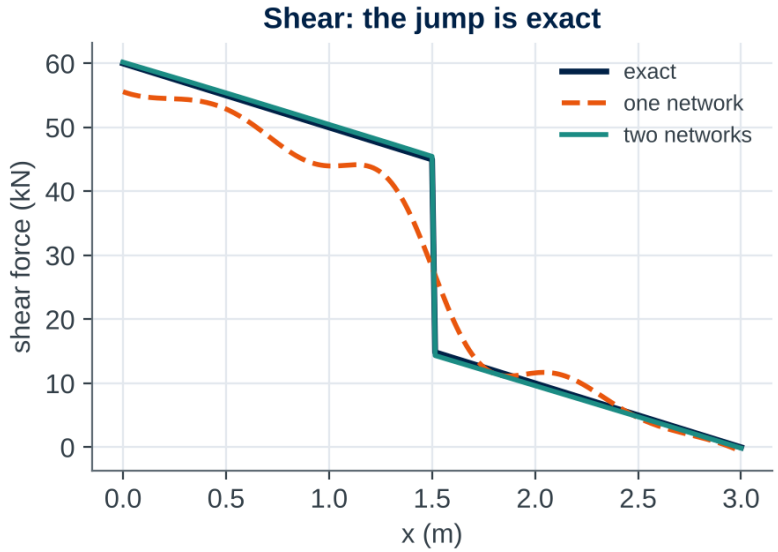
- Two networks, each smooth.
- w, w', w" continuous.
- Shear jumps by P.
- No Gaussian, no spike.

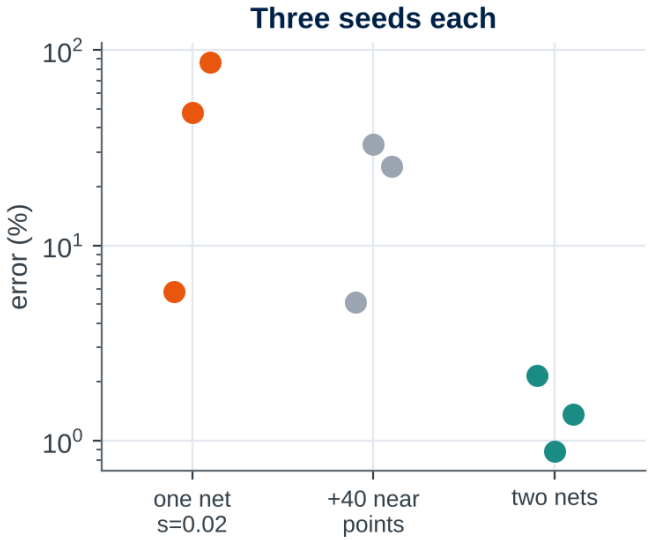
after Jagtap and Karniadakis 2020 (XPINN); Babanagar and Sheil, ISARC 2026.

Domain decomposition in code


```
l0, l1, l2, l3, _ = derivs(netL, xa)                                  # left net
r0, r1, r2, r3, _ = derivs(netR, xa)                                  # right net
L_int = ((l0 - r0)**2 + (l1 - r1)**2 + (l2 - r2)**2 + (r3 - l3 - P)**2).sum()      # TODO 1
loss  = L_phys + 10 * (L_bc + L_int)
```





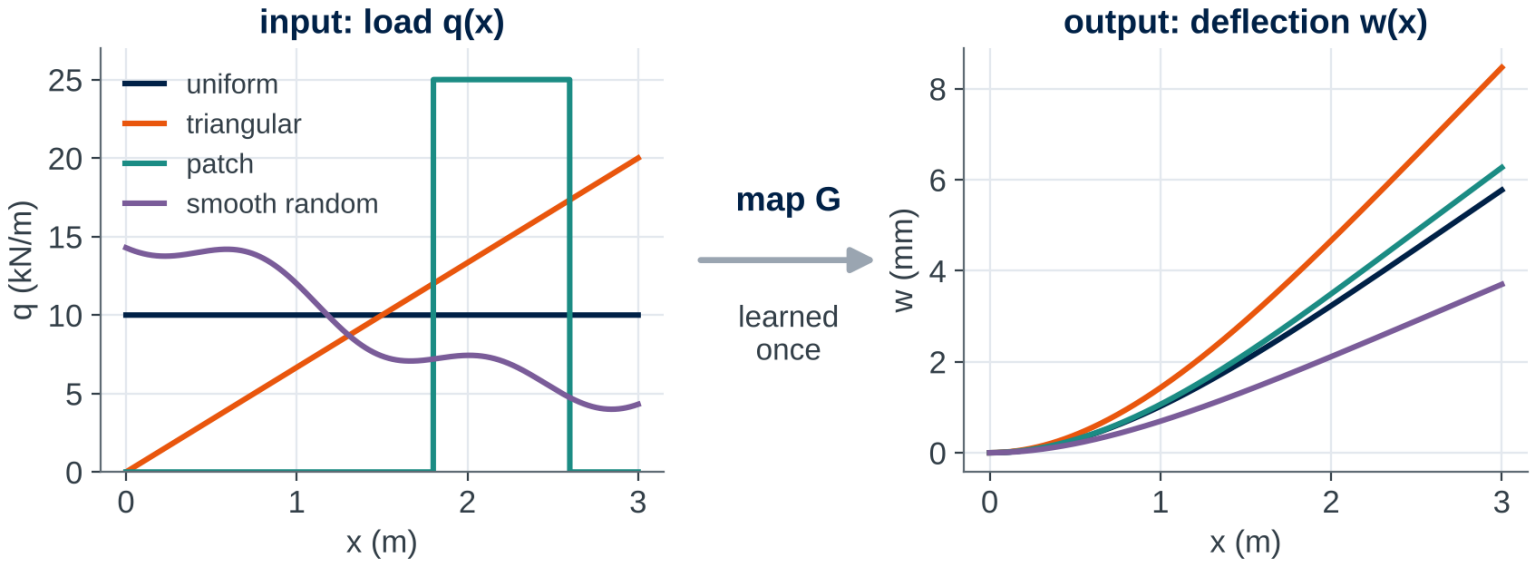


Two networks: 0.9–2.2 % error over three seeds; one network at s = 0.02: 6–87 %.

UNIVERSITY OF CAMBRIDGE

A PINN solves one problem; design needs many

Concept



- Our PINN: one load, one section, one set of supports. A new load case means a new training run.
- Design, reliability analysis and monitoring ask for thousands of load cases.
- A neural operator learns the whole map from (load, response) pairs once, then answers a new load in one forward pass.

$w(x) = \int G(x, \xi)\, q(\xi)\, d\xi$

The beam is already an operator: for the cantilever G is known (a Green's function), and this figure was computed with it. For cracking, soil plasticity or consolidation G has no closed form, so we learn it.

Exact cantilever solutions (IPE 300, L = 3 m, EI = 1.755 × 10^7 N·m²): uniform 10 kN/m gives a tip deflection of 5.77 mm.

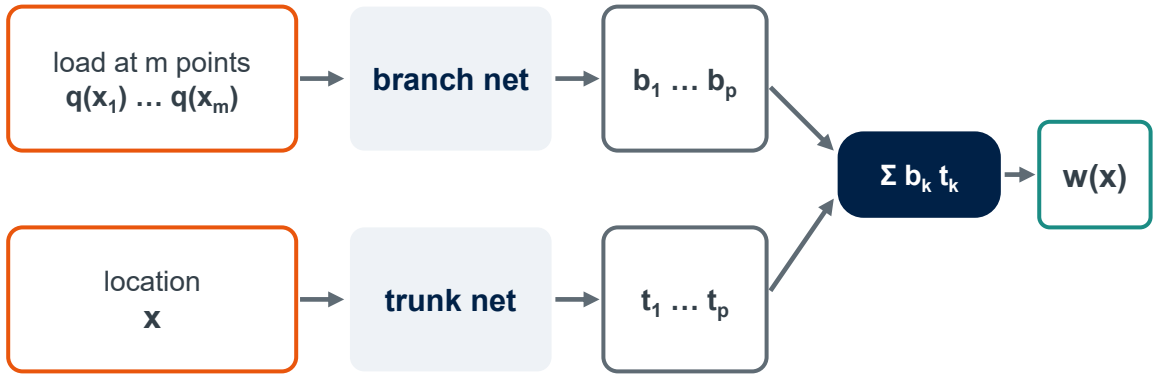
From one solution to the solution operator.

Neural operators in one picture

Concept

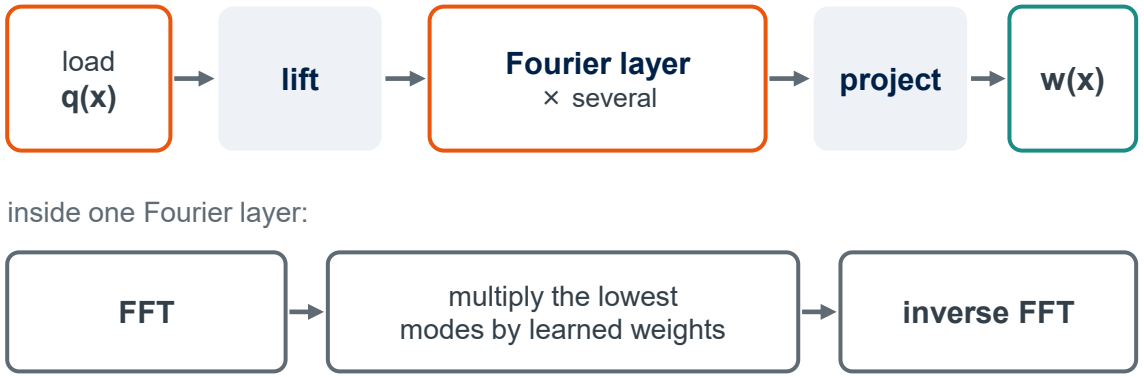
DeepONet

the branch net reads the load, the trunk net reads the location



Fourier neural operator (FNO)

works on the load as a function: it can be evaluated at other resolutions



inside one Fourier layer:

Training data

Pairs of load and response from finite element runs or tests. Physics-informed versions (PI-DeepONet, PINO) add the equation to the loss and need fewer pairs.

What you gain

No retraining for a new load: one forward pass. FNO is resolution-independent. Useful for design, reliability and monitoring.

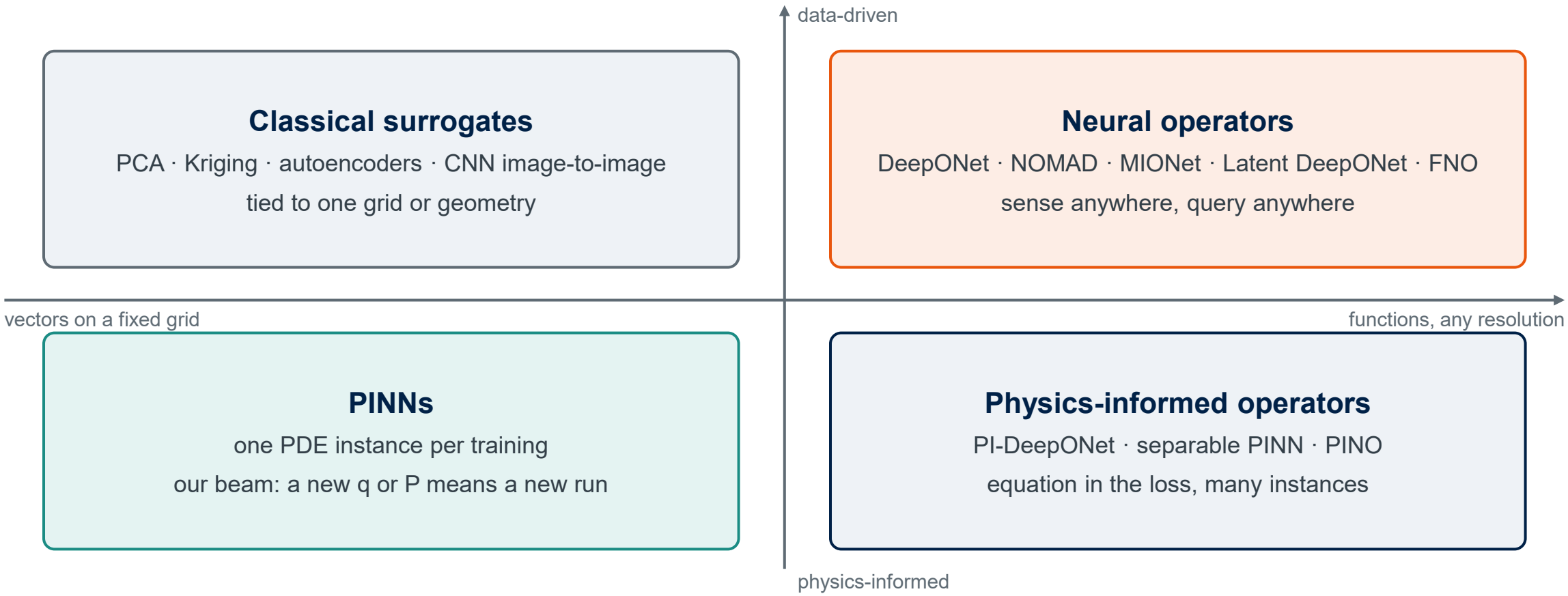
What to watch

Many solves are needed up front, and the answer can be trusted only for loads like those it was trained on.

Lu et al. 2021 (DeepONet); Li et al. 2021 (FNO); Wang, Wang & Perdikaris 2021 (PI-DeepONet); Li et al. 2024 (PINO); review: Kovachki et al. 2023.

Operators answer “what if the load changes?”; PINNs answer “what is the solution for this load?”

One problem or a family of problems?



A PINN learns one problem. If the load, the section or the supports change, we train again.

BLOCK 9

Wrap-up

summary, checklist, next steps

What we learned

- Data alone can fit the sensors and still get the moment wrong; physics constrains it.
- A PINN is a network plus a physics loss. Derivatives come from automatic differentiation; no mesh.
- Scaling and loss weights decide whether training works at all; adaptive weighting automates the choice.
- Inverse problems: the load is one more trainable parameter.
- Sharp features (a point load) trigger spectral bias; domain decomposition makes the load an interface condition.
- A good fit is not proof of the right parameters: check what the sensors can identify.
- A PINN solves one problem; neural operators learn the map from load to response for a whole family.

When should you use a PINN?

GOOD FIT

Use a PINN when

The physics is known and the data are sparse or noisy.

Parameters or loads are unknown (inverse problem).

You need a smooth, differentiable surrogate.

Meshing is awkward or the geometry changes.

POOR FIT

Use a classical solver when

The forward problem is well posed and fast to solve.

The solution has sharp features and you have no time to tune.

You need a certified accuracy.

There are no unknowns to find.

A PINN complements a simulation; it does not replace it.

References

- Babanagar, N. and Sheil, B. (2026). ISARC 2026 (cantilever case, PINN versus data-only, domain decomposition).
- Cicirello, A. (2024). Physics-enhanced machine learning: a position paper for dynamical systems investigations. J. Phys.: Conf. Ser. 2909, 012034.
- Faroughi, S. et al. (2024). Physics-guided, physics-informed, and physics-encoded neural networks in scientific computing. J. Comput. Inf. Sci. Eng. 24, 040802.
- Raissi, M., Perdikaris, P. and Karniadakis, G. E. (2019). Physics-informed neural networks. J. Comput. Phys. 378, 686–707.
- [1] Sahin, T. et al. (2024). Towards a hybrid digital twin: PINNs as surrogate model of a reinforced concrete beam. arXiv:2405.08406.
- [2] Gong, W. et al. (2024). Prediction of stratified ground consolidation via a PINN utilizing short-term excess pore water pressure monitoring data. Comput.-Aided Civ. Infrastruct. Eng.
- [3] He, Q. et al. (2020). PINNs for multiphysics data assimilation with application to subsurface transport. Adv. Water Resour.
- [4] Huang, Z. et al. (2026). PINN based inversion and prediction of natural chloride diffusion in uncracked and cracked concrete systems. Comput. Struct. 322, 108120.
- [5] Rahman, M. A., Zhang, T. and Lu, Y. (2024). PINN-CHK: PINN for high-fidelity prediction of early-age cement hydration kinetics. Neural Comput. Appl. 36(22), 13665–13687.
- Bekele, Y. W. (2021). Physics-informed deep learning for one-dimensional consolidation. J. Rock Mech. Geotech. Eng. 13.
- Haghighat, E. et al. (2021). A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics. Comput. Methods Appl. Mech. Eng. 379, 113741.
- Niaki, S. A. et al. (2021). PINN for modelling the thermochemical curing process of composite-tool systems during manufacture. Comput. Methods Appl. Mech. Eng. 384, 113959.
- Lu, L. et al. (2021). DeepXDE. SIAM Rev. 63, 208–228.
- Rahaman, N. et al. (2019). On the spectral bias of neural networks. ICML.
- Wang, S., Teng, Y. and Perdikaris, P. (2021). Understanding and mitigating gradient flow pathologies in PINNs. SIAM J. Sci. Comput. 43, A3055–A3081.
- Wang, S., Yu, X. and Perdikaris, P. (2022). When and why PINNs fail to train: a neural tangent kernel perspective. J. Comput. Phys. 449, 110768.
- Wang, S., Wang, H. and Perdikaris, P. (2021). On the eigenvector bias of Fourier feature networks. Comput. Methods Appl. Mech. Eng. 384, 113938.
- McClenny, L. D. and Braga-Neto, U. (2023). Self-adaptive physics-informed neural networks. J. Comput. Phys. 474, 111722.
- Jagtap, A. D. and Karniadakis, G. E. (2020). Extended PINNs (XPINN). Commun. Comput. Phys. 28, 2002–2041.
- Kharazmi, E., Zhang, Z. and Karniadakis, G. E. (2021). hp-VPINNs: variational PINNs with domain decomposition. Comput. Methods Appl. Mech. Eng. 374, 113547.
- Tancik, M. et al. (2020). Fourier features let networks learn high-frequency functions in low dimensional domains. NeurIPS 33.
- Wang, S., Sankaran, S. and Perdikaris, P. (2024). Respecting causality for training physics-informed neural networks. Comput. Methods Appl. Mech. Eng. 421, 116813.

References (continued): remedies and operators

- Bischof, R. and Kraus, M. A. (2025). Multi-objective loss balancing for physics-informed deep learning. Comput. Methods Appl. Mech. Eng. 439, 117914.
- Wu, C., Zhu, M., Tan, Q., Kartha, Y. and Lu, L. (2023). A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. Comput. Methods Appl. Mech. Eng. 403, 115671.
- Sukumar, N. and Srivastava, A. (2022). Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. Comput. Methods Appl. Mech. Eng. 389, 114333.
- Rezaei, S., Harandi, A., Moeineddin, A., Xu, B.-X. and Reese, S. (2022). A mixed formulation for physics-informed neural networks as a potential solver for engineering problems in heterogeneous domains. Comput. Methods Appl. Mech. Eng. 401, 115616.
- Jagtap, A. D., Kharazmi, E. and Karniadakis, G. E. (2020). Conservative physics-informed neural networks on discrete domains for conservation laws. Comput. Methods Appl. Mech. Eng. 365, 113028.
- Yang, L., Meng, X. and Karniadakis, G. E. (2021). B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data. J. Comput. Phys. 425, 109913.
- Wang, S., Sankaran, S., Wang, H. and Perdikaris, P. (2023). An expert's guide to training physics-informed neural networks. arXiv:2308.08468.
- Hao, Z. et al. (2024). PINNacle: a comprehensive benchmark of physics-informed neural networks for solving PDEs. NeurIPS 37 (Datasets and Benchmarks).
- Lu, L., Jin, P., Pang, G., Zhang, Z. and Karniadakis, G. E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nat. Mach. Intell. 3, 218–229.
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A. and Anandkumar, A. (2021). Fourier neural operator for parametric partial differential equations. ICLR 2021.
- Wang, S., Wang, H. and Perdikaris, P. (2021). Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. Sci. Adv. 7, eabi8605.
- Li, Z., Zheng, H., Kovachki, N., Jin, D., Chen, H., Liu, B., Azizzadenesheli, K. and Anandkumar, A. (2024). Physics-informed neural operator for learning partial differential equations. ACM/IMS J. Data Sci., doi:10.1145/3648506.
- Kovachki, N. et al. (2023). Neural operator: learning maps between function spaces with applications to PDEs. J. Mach. Learn. Res. 24(89), 1–97.
- Cuomo, S. et al. (2022). Scientific machine learning through physics-informed neural networks: where we are and what's next. J. Sci. Comput. 92, 88.